

Trabajo de Fin de Máster

Evaluación de LLMs para la Generación de Ontologías en Bases de Datos Genéticas

Máster Universitario en Inteligencia Artificial

Facultad de Informática · Universidad de Murcia

Documento de metodología

Autor: Francisco Sáez

Versión: 11.0

Fecha: 9 de mayo de 2026

Tutor: *[Nombre del tutor o tutora]*

Abstract

The manual construction of a biomedical ontology requires weeks of expert effort. Two ontologists rarely converge on the same schema. The operational question addressed in this Master's Thesis is: given a CSV from a genetic database, how far can a Large Language Model go on its own, and where is it worth supporting it?

This work develops a benchmark with four strategies (zero-shot, controlled vocabulary, RAG, multi-agent OntoGenix pipeline), three models (gpt-4o, Llama 3.1 8B, Qwen 2.5 Coder 7B) and four cis-regulatory databases (FANTOM5, dbSUPER, HACER, DiseaseEnhancer). Around 140 ontologies are generated and evaluated with OQuaRE — the normative framework based on ISO/IEC 25000 — integrating the HermiT reasoner via owlready2. The keyword-based RAG is also replaced by a real semantic RAG, indexed in Neo4j with sentence-transformers embeddings.

The multi-agent E4 pipeline (OntoGenix) yields the greatest absolute richness — ontologies between 3.7× and 5.3× denser than E1–E3 baselines (mean 4.4×) — yet receives lower OQuaRE Global scores because the rubric assigns lower values to higher structural complexity. The semantic RAG raises the E3 score with gpt-4o from 2.91 to 4.20, a +44 % gain driven by a sixfold improvement in relational operability. When the same RAG is applied to Llama 3.1 8B, the effect inverts: the small model saturates with rich context and half of the runs fail to validate. Reducing top_k from 5 to 2 closes 84 % of the gap relative to gpt-4o without changing the model. Each strategy exhibits a characteristic error pattern: Llama omits the prefix colon, Qwen mixes Turtle with RDF/XML, gpt-4o makes no systematic errors.

The thesis presents three replicable findings: an empirical calibration guide for RAG by model size, a set of mechanical fixes that recover 100 % of recoverable TTLs without re-prompting the model, and a reproducibility framework aligned with the replication crisis in machine learning research.

Palabras clave / Keywords

Palabras clave: ontologías biomédicas; modelos de lenguaje grandes; generación de ontologías; RAG semántico; OQuaRE; OntoGenix; reproducibilidad; dominio cis-regulatorio; OWL; HermiT.

Keywords: biomedical ontologies; Large Language Models; ontology generation; semantic Retrieval-Augmented Generation; OQuaRE; OntoGenix; reproducibility; cis-regulatory domain; OWL; HermiT.

Lista de tablas

Tabla 1. Configuración global del experimento (campo y valor por defecto)

Tabla 2. Pipeline replicada de OntoGenix, paso a paso

Tabla 3. Adaptaciones técnicas de OntoGenix al entorno del TFM

Tabla 4. Parámetros de generación comunes (E1–E3)

Tabla 5. Corpus de datos del dominio cis-regulatorio utilizado

Tabla 6. Estrategias de muestreo evaluadas (cabecera, aleatorio, estratificado, diversidad)

Tabla 7. Resultados del análisis de sensibilidad por (modelo × experimento × estrategia)

Tabla 8. Roles de los tres agentes de la pipeline OntoGenix

Tabla 9. Métricas estructurales de evaluación calculadas sobre cada ontología

Tabla 10. Efecto del post-procesado mecánico sobre la validez sintáctica

Tabla 11. Tabla resumen consolidada — métricas absolutas por experimento y BBDD

Tabla 12. Rúbricas OQuaRE (Duque-Ramos et al., 2014) por sub-característica

Tabla 13. OQuaRE Global por modelo y experimento (escala 1-5)

Tabla 14. Validez sintáctica cross-model tras post-procesado

Tabla 15. OQuaRE Global por celda — modelo × experimento × BBDD

Tabla 16. Comparativa E4 OntoGenix con gpt-4o frente a Llama 3.1 8B

Tabla 17. Arquitectura del RAG semántico (annotationRAG): componentes y responsabilidades

Tabla 18. Descomposición OQuaRE: RAG por keywords frente a RAG semántico (gpt-4o)

Tabla 19. Coste de la integración del RAG semántico (recursos y dependencias)

Tabla 20. Cuadro 2×2: efecto del backend RAG en gpt-4o y Llama 3.1 8B

Tabla 21. Configuraciones de calibración del RAG (C1, C2, C3)

Tabla 22. Resultados de la calibración del RAG sobre Llama 3.1 8B en E3

Tabla 23. Recomendación operativa de top_k según el tamaño del modelo

Tabla 24. Pila de herramientas y dependencias del banco experimental

Tabla 25. Amenazas específicas a la reproducibilidad en investigación con LLMs

Tabla 26. Tres niveles de garantía de reproducibilidad (R1 · R2 · R3)

Tabla 27. Resumen de tests estadísticos formales (Bonferroni-Holm)

Tabla 28. Análisis de potencia para OQuaRE Global

Tabla 29. Fidelidad léxico-semántica al dominio cisreg por celda

Tabla 30. Consistencia inter-BBDD del core semántico (Jaccard medio)

Tabla 31. Cobertura de preguntas de competencia por celda (n CQ satisfechas de 15)

Tabla 32. Estimación de tokens y coste proyectado para el barrido del tamaño muestral

Tabla 33. Resultados del barrido del tamaño muestral (media sobre 3 corridas)

Lista de figuras

Figura 1. Validez sintáctica (rdflib) por experimento, antes y después del post-procesado

Figura 2. Media de triples por ontología desglosada por experimento y base de datos

Figura 3. Distribución del número de rdfs:label por ontología

Figura 4. Diagrama radar normalizado de las métricas absolutas por experimento

Figura 5. Puntuación OQuaRE Global (escala 1-5) por experimento y modelo

Figura 6. Sub-características OQuaRE — promedio por modelo

Figura 7. Porcentaje de ontologías que cargan en owlready2 (load_ok) por experimento y modelo

Figura 8. Literales con tipo numérico espurio detectados y saneados

Figura 9. Análisis de sensibilidad al muestreo (4 estrategias × 2 BBDD × 2 modelos)

Figura 10. Comparativa OQuaRE entre el RAG por keywords y el RAG semántico (gpt-4o)

Figura 11. Cuadro 2×2: efecto del backend RAG sobre gpt-4o y Llama 3.1 8B

Figura 12. Calibración del RAG semántico para Llama 3.1 8B en E3

Figura 13. Fidelidad léxico-semántica al dominio cisreg (recall y canonical_ratio)

Figura 14. Sensibilidad al tamaño muestral en E3 RAG semántico

1. Introducción y objetivos

Los modelos de lenguaje grandes (LLM) son capaces de producir ontologías serializadas en Turtle, con un rendimiento que varía considerablemente en función de la estrategia de prompting empleada. La literatura propone cuatro estrategias principales: zero-shot, vocabulario controlado, generación aumentada por recuperación (RAG) y pipelines multi-agente. La aproximación más sofisticada (OntoGenix, Val Calvo et al., 2024) emplea tres agentes secuenciales y reporta ontologías más ricas que el prompting directo. Las afirmaciones publicadas, sin embargo, resultan difíciles de comparar porque cada trabajo emplea su propio conjunto de datos, su propia métrica y, habitualmente, un único modelo comercial cerrado.

1.1. Objetivos específicos (alcanzados)

Los objetivos del trabajo se enumeran a continuación. El conjunto responde a las hipótesis de investigación formuladas en §1.2 y se evalúa empíricamente en los Apartados 8–10.

- **Banco de pruebas reproducible.** Construir un banco experimental que cubra cuatro experimentos (E1–E4), cuatro bases de datos del dominio cisreg (FANTOM5, dbSUPER, HACER, DiseaseEnhancer) y tres semillas (42, 43, 44), produciendo una n teórica de 144 corridas en el experimento principal y aproximadamente 144 corridas adicionales en los sub-experimentos.
- **Replicación funcional de OntoGenix.** Ejecutar la pipeline multi-agente de Val Calvo et al. (2024) sobre las cuatro bases de datos sin la GUI PyQt5 original, con tres adaptaciones técnicas documentadas (bypass del Searcher, bypass del paso SERP, snapshot del modelo). Detalle en §3.4.5.
- **Métricas objetivas y normativas.** Adoptar OQuaRE (Duque-Ramos et al., 2014) como marco normativo basado en ISO/IEC 25000, implementado contra el razonador HermiT vía owlready2. Complementar con métricas de fidelidad léxico-semántica al dominio (§9.5) y validación funcional por preguntas de competencia (§9.6).
- **Documentación de los patrones de error.** Caracterizar los errores sintácticos y semánticos sistemáticos de cada modelo y estrategia, evaluando la recuperabilidad mecánica vía post-procesado determinista.

- **Comparativa cross-model.** Comparar el modelo cerrado de referencia (gpt-4o) con dos modelos open-source compactos (Llama 3.1 8B, Qwen 2.5 Coder 7B) sobre el mismo banco, manteniendo prompts y semillas constantes.
- **Integración de un RAG semántico real.** Sustituir el RAG por keyword-match estático del experimento original por un microservicio basado en embeddings densos (sentence-transformers) e índice vectorial Neo4j sobre 21 ontologías biomédicas. Calibrar sus parámetros (top_k, score_thr, max_chars) por tamaño de modelo (§10.7.7).

1.2. Hipótesis de investigación

Los objetivos anteriores se articulan en torno a cuatro hipótesis principales que el banco experimental pone a prueba:

- H1.** Las pipelines multi-agente (E4 · OntoGenix) producen ontologías estructuralmente más ricas en número de triples, clases, propiedades y etiquetas que las estrategias de prompting directo (E1 zero-shot, E2 vocabulario controlado, E3 RAG).
- H2.** La sustitución del RAG por keyword-match por un RAG semántico real basado en embeddings y vector index incrementa significativamente la puntuación OQuaRE Global, al menos para modelos con suficiente capacidad de absorción de contexto.
- H3.** Existe una interacción entre tamaño del modelo y configuración del retrieval: los modelos compactos (Llama 3.1 8B) se saturan con la configuración por defecto del RAG semántico, y valores reducidos de top_k recuperan parte del rendimiento.
- H4.** El post-procesado mecánico determinista permite rescatar la totalidad de los outputs sintácticamente recuperables sin invocaciones adicionales al modelo, separando el problema sintáctico (parser) del problema semántico (modelado).

Estas hipótesis se evalúan cuantitativamente con OQuaRE, la métrica complementaria de fidelidad léxico-semántica (§9.5) y la validación funcional mediante preguntas de competencia (§9.6); el alcance del trabajo y las limitaciones que acotan estas hipótesis se discuten en §13.5.

2. Estado del arte y marco teórico

Este apartado proporciona el marco teórico y revisa la literatura reciente sobre los cuatro ejes en los que se inscribe el presente trabajo: la representación formal de ontologías OWL 2, la generación automática de ontologías mediante modelos de lenguaje grandes, los sistemas de generación aumentada por recuperación con componentes semánticos, y la evaluación normativa de la calidad ontológica. El apartado cierra con una discusión del problema general de la reproducibilidad en investigación con LLMs, marco que justifica las decisiones metodológicas adoptadas en §3.

2.1. Ontologías biomédicas y OWL 2

Una ontología, en el sentido de Studer, Benjamins y Fensel (1998), es una especificación formal y explícita de una conceptualización compartida. En la web semántica se expresan habitualmente en OWL 2 (Motik, Patel-Schneider y Parsia, 2012), serializadas en Turtle (W3C, 2014), con una sintaxis

orientada a triples sujeto-predicado-objeto. Los elementos fundamentales son las clases (owl:Class), las propiedades de objeto (owl:ObjectProperty), las propiedades de datos (owl:DatatypeProperty), las restricciones (owl:Restriction) y los axiomas subsumptivos (rdfs:subClassOf). OWL 2 define tres perfiles computacionalmente tratables: EL para razonamiento polinómico en ontologías biomédicas con gran número de clases, RL para razonamiento basado en reglas, y QL para integración con bases de datos relacionales (Cuenca-Grau et al., 2008). El perfil DL, más expresivo, mantiene la completitud y decidibilidad sobre la lógica de descripción SROIQ y es el utilizado por razonadores como HermiT (Glimm et al., 2014) y Pellet.

En el dominio biomédico, las ontologías representan jerarquías de entidades biológicas, propiedades genómicas y relaciones funcionales. Las más establecidas (GO, ChEBI, OBI, SNOMED-CT) cuentan con miles de clases curadas manualmente. Las ontologías más jóvenes, como las del dominio cis-regulatorio (CRM, enhancer-gene regulation), aún están en desarrollo activo y carecen de la cobertura de las primeras, lo que abre espacio para sistemas automáticos o semi-automáticos que asistan al experto humano en la fase de modelado inicial.

2.2. Generación de ontologías con modelos de lenguaje grandes

La aplicación de LLMs a tareas de ingeniería ontológica ha experimentado un crecimiento sostenido desde 2022. Cuatro paradigmas dominan la literatura.

Generación zero-shot. La estrategia más simple consiste en pedir al modelo, sin ejemplos, que produzca una ontología a partir de una descripción del dominio o de un esquema tabular. Funcionan razonablemente con GPT-4 y modelos comparables (Meyer et al., 2023), pero la salida es estilísticamente inconsistente entre ejecuciones y propensa a errores de sintaxis Turtle. La estrategia se complementa frecuentemente con few-shot, donde se proporcionan uno o varios ejemplos de pares (input, ontología) antes de la consulta real, mejorando la consistencia estilística a costa de aumentar la longitud del prompt (Brown et al., 2020).

Vocabulario controlado. El modelo recibe una lista cerrada de términos ontológicos —clases y propiedades canónicas del dominio— y se le pide que aliñe la ontología generada con ese vocabulario mediante skos:exactMatch o skos:closeMatch (Mateiu y Groza, 2023). Es una variante intermedia entre zero-shot y RAG: reduce la dispersión léxica pero limita la riqueza del esquema resultante a las relaciones que el vocabulario provee.

Generación aumentada por recuperación (RAG). Se recuperan fragmentos de ontologías similares y se inyectan en el prompt como contexto. La primera generación de sistemas (Lewis et al., 2020; aplicado a ontologías por Norouzi et al., 2024) utilizaba retrieval léxico simple (BM25, TF-IDF, keyword-match). La generación reciente sustituye esta capa por embeddings densos generados con sentence-transformers (Reimers y Gurevych, 2019), indexados en bases de datos vectoriales como Pinecone, Weaviate o Neo4j con índice vectorial nativo. En el dominio biomédico, embeddings especializados como SapBERT (Liu et al., 2021), BioBERT (Lee et al., 2020) y BiomedBERT (Gu et al., 2021) reportan mejor recuperación que los embeddings de propósito general. Caufield et al. (2023) introducen SPIRES, un sistema de extracción zero-shot basado en LLMs que populariza el patrón de RAG aplicado a ontology learning.

Pipelines multi-agente. Conjuntos de LLMs especializados se coordinan para descomponer la tarea: un agente planifica, otro construye el TBox, un tercero mapea instancias del CSV al ABox. OntoGenix (Val Calvo, Egaña Aranguren y Fernández Breis, 2024) es el referente actual en el dominio biomédico cisreg. Otros sistemas relacionados son OntoGPT (Caufield et al., 2024), que integra grounding contra ontologías de referencia, y KGGen (Kommineni et al., 2024), enfocado a construir grafos de conocimiento generales. La complejidad orquestal incrementa la riqueza estructural de la salida a costa de un coste computacional muy superior.

2.3. RAG semántico y embeddings biomédicos

El RAG semántico para ontology engineering se apoya en tres componentes: un modelo de embeddings que mapea cada fragmento ontológico a un vector denso; un índice vectorial que soporta búsqueda aproximada por similitud coseno; y una capa de orquestación que construye el contexto inyectado al LLM con los k fragmentos más cercanos a la consulta. El tamaño del vector (dimensión del embedding), la granularidad del chunk indexado (axioma, clase, módulo o esquema completo), el valor de k y el umbral de score de corte son los parámetros principales del sistema. El presente trabajo emplea annotationRAG, un microservicio FastAPI desarrollado por el grupo Tecnomod (Universidad de Murcia) que combina sentence-transformers con índice vectorial Neo4j 5.26 sobre 21 ontologías del dominio cis-regulatorio y vocabularios auxiliares.

La calidad del embedding determina qué se recupera. all-MiniLM-L6-v2 (Wang et al., 2020) es la opción por defecto por su tamaño compacto (22 MB, 384 dimensiones) y latencia muy baja, pero no está entrenado específicamente para terminología biomédica. Embeddings biomédicos especializados como SapBERT (Liu et al., 2021) o BiomedBERT (Gu et al., 2021) han demostrado mejor desempeño en tareas como entity linking y ontology alignment. La elección entre embeddings genéricos y especializados implica un compromiso entre fidelidad de dominio, coste computacional y disponibilidad de hardware (los modelos biomédicos suelen requerir GPU para latencia aceptable).

2.4. Evaluación automática de la calidad ontológica

La evaluación de ontologías generadas automáticamente combina tres estrategias principales: validación sintáctica (parse-OK contra un razonador o parser RDF), métricas estructurales (cohesión, modularidad, complejidad) y métricas semánticas (fidelidad al dominio, riqueza axiomática). La validación sintáctica es trivial pero esencial: una ontología que no parsea no puede evaluarse posteriormente. Las métricas estructurales tradicionales (Tartir et al., 2005; Vrandečić, 2010) miden propiedades grafo-teóricas como número de clases, número de propiedades, profundidad de la jerarquía y densidad relacional.

OQuaRE (Duque-Ramos et al., 2014) es el marco normativo más establecido para evaluación automática. Adapta ISO/IEC 25000 (SQuaRE, originalmente para software) al dominio ontológico, definiendo cinco sub-características —Structural, Modularity, Reusability, Operability, Reliability— operacionalizadas mediante once métricas estructurales con rúbricas publicadas en escala 1–5. OQuaRE no captura corrección biomédica de las relaciones inferidas, lo que requiere validación experta sobre una muestra (Bandeira et al., 2016) o, alternativamente, métricas de fidelidad léxica

como las introducidas en este trabajo (§9.5) y de validación funcional mediante preguntas de competencia en SPARQL (Grüninger y Fox, 1995). Existen además recomendadores no normativos como NCBO Recommender (Martínez-Romero et al., 2017) y BioPortal Quality Score (Ochs et al., 2017), enfocados a la evaluación de ontologías biomédicas publicadas.

2.5. Reproducibilidad y crisis de la replicación

La denominada «crisis de la replicación», identificada inicialmente en psicología (Open Science Collaboration, 2015) y en investigación preclínica (Begley y Ellis, 2012), se ha extendido a la inteligencia artificial. Hutson (2018) estimó que entre el 70 % y el 90 % de los experimentos publicados en grandes conferencias de IA no son reproducibles con la información proporcionada. Gundersen y Kjensmo (2018) sistematizaron el problema clasificando los experimentos en tres niveles —R1 estricta, R2 estadística, R3 metodológica— en función de las garantías que el procedimiento ofrece. Pineau et al. (2021) reportaron los resultados del programa de reproducibilidad de NeurIPS 2019 y propusieron un checklist que ha sido adoptado por las principales conferencias.

La investigación con LLMs introduce amenazas específicas: (i) el determinismo bit-a-bit no se garantiza ni con temperature=0 y seed fija, en parte por la concurrencia interna del proveedor (Henderson et al., 2018); (ii) los alias de modelo (p. ej. gpt-4o) son móviles y apuntan a snapshots distintos en función del momento de la llamada; (iii) las quantizaciones de modelos open-source (Q4_K_M, Q5_K_S) pueden cambiar entre re-descargas; y (iv) los embeddings publicados se actualizan sin notificación. El presente trabajo encuadra estas amenazas en la taxonomía R1/R2/R3 (§12) y explicita las limitaciones de reproducibilidad de cada componente.

3. Metodología

Este apartado describe el banco experimental, las herramientas y los datos empleados para la evaluación comparativa de las cuatro estrategias de generación. La descripción se organiza en seis sub-secciones: conjuntos de datos de validación (§3.1), ontologías de referencia del dominio cis-regulatorio (§3.2), modelos LLM utilizados (§3.3), estrategias de generación con sus parámetros y prompts (§3.4), sistema de evaluación (§3.5) y post-procesado mecánico (§3.6).

3.1. Conjuntos de datos de validación

El banco experimental utiliza cuatro bases de datos del dominio cis-regulatorio humano. Las cuatro comparten un mismo esquema columnar (treinta columnas canónicas obtenidas tras un pre-procesado de homogeneización) que cubre: identificador del módulo regulador, coordenadas genómicas en dos ensamblajes (hg19 y hg38), score de confianza, referencia PubMed, biosample, método experimental, tipo de regulación, fuente, genes diana, factores de transcripción, enfermedades asociadas y mutaciones registradas. Sus características distintivas, junto con la finalidad biológica de cada una, se resumen a continuación.

FANTOM5 (Functional Annotation of the Mammalian Genome 5; Andersson et al., 2014). Atlas de promotores y enhancers basado en CAGE-seq, cubre 1.829 muestras de tejidos y líneas celulares humanas. En el banco se utiliza el subconjunto de enhancers activos (~37 000 entradas).

dbSUPER (Database of Super-Enhancers; Khan y Zhang, 2016). Catálogo de super-enhancers anotados manualmente sobre 102 tejidos y tipos celulares humanos. Aporta la dimensión de regulación regional extendida (super-enhancer vs enhancer convencional).

HACER (Human ACTIVE Enhancer to gene Regulation; Wang et al., 2019). Asocia enhancers activos con sus genes diana mediante eQTL y co-expresión. Cubre 1 408 muestras.

DiseaseEnhancer (Zhang et al., 2018). Anota enhancers asociados a enfermedades humanas mediante variantes funcionales y evidencias experimentales reportadas en literatura.

La uniformidad del esquema columnar entre las cuatro bases de datos es un aspecto relevante para la interpretación de los resultados: dado que las cuatro presentan las mismas treinta columnas, cualquier variación inter-BBDD en las ontologías generadas es atribuible al contenido de las celdas o al ruido inter-corrida, no a diferencias estructurales del input. Esto se discute con detalle en §10.8.

3.2. Ontologías cisreg de referencia

Las ontologías de referencia del dominio cis-regulatorio utilizadas en este trabajo proceden del proyecto BioGateway (Universidad de Murcia) y modelan ocho aspectos complementarios:

- `crm.ttl` — esquema de módulos cis-reguladores básicos (CRM como entidad principal con coordenadas, taxón y ensamblaje).
- `crm_example.ttl` — instanciación de `crm.ttl` con valores concretos para ilustrar el modelado.
- `crm2gene.ttl` — relaciones entre CRM y genes diana.
- `crm2gene_example.ttl` — instancias de la relación CRM→gen.
- `crm2phen.ttl` — relaciones entre CRM y fenotipos asociados.
- `crm2phen_example.ttl` — instancias.
- `crm2tfac.ttl` — relaciones entre CRM y factores de transcripción.
- `crm2tfac_example.ttl` — instancias.

Los esquemas declaran URIs canónicas bajo el espacio de nombres `http://rdf.biogateway.eu/`, que se reutilizan en las ontologías generadas mediante alineamiento explícito (`skos:exactMatch` / `skos:closeMatch`). Un fragmento ilustrativo del esquema `crm` es el siguiente:

```
hcrm:crm_ID rdfs:type owl:Class; rdfs:subClassOf obo:SO_0000727; skos:prefLabel "crm/crm_ID";
obo:RO_0002162 obo:NCBITaxon_9606; obo:BFO_0000050 nuccore:chr_ID; obo:GENO_0000895
"start_position"^^xsd:integer; obo:GENO_0000894 "end_position"^^xsd:integer.
```

Estas ocho ontologías constituyen también el gold set utilizado en la métrica de fidelidad léxica (§9.5), del que se extraen 88 URIs canónicas únicas como referencia.

3.3. Modelos LLM utilizados

La evaluación se realiza sobre tres modelos de capacidades muy distintas, seleccionados para cubrir el rango operativo realista de un equipo biomédico:

- gpt-4o (snapshot 2024-05-13; OpenAI). Modelo cerrado de propósito general, accesible vía API. Representa la frontera de capacidad disponible comercialmente.
- Llama 3.1 8B Instruct (Meta). Modelo abierto generalista, accesible localmente vía Ollama con cuantización Q4_K_M. Representa la baseline open-source de tamaño compacto desplegable en hardware de consumo.
- Qwen 2.5 Coder 7B (Alibaba). Modelo abierto especializado en generación de código, accesible vía Ollama. Permite contrastar si la especialización en código se traduce en mejor sintaxis Turtle.

Los tres modelos se invocan a través de una API compatible con OpenAI: OpenAI directamente para gpt-4o, y Ollama (puerto local 11434) con wrapper compatible para los dos modelos abiertos. Los parámetros de generación son comunes (temperature, top_p, seed) y se detallan en §3.4.

3.4. Estrategias de generación

Las cuatro estrategias evaluadas comparten un mismo input —una muestra de 25 filas del CSV original en formato TSV, precedida por cinco líneas de comentario con metadatos— y una misma salida esperada —un archivo Turtle válido que represente la ontología generada—. La diferencia entre estrategias reside en cómo se inyecta el contexto y en cómo se orquestan las llamadas al LLM.

3.4.1. Prompt común y formato del CSV

El system prompt común a E1–E3 establece el rol del modelo como ontólogo experto en grafos biomédicos. El user prompt incluye el nombre de la base de datos, su descripción funcional, la muestra de 25 filas y un bloque variable de contexto (vacío para E1, vocabulario para E2, fragmentos ontológicos para E3). En E4 (OntoGenix), los tres agentes utilizan plantillas externas (LlmPlanner, LlmOntology, LlmOntoMapper) que se invocan secuencialmente con el CSV completo. La muestra se conserva como texto TSV plano —sin construir un feature vector intermedio— para preservar la estructura tabular original que el modelo necesita para identificar entidades y relaciones.

3.4.2. E1 — Generación zero-shot

El modelo recibe la muestra del CSV y la instrucción genérica de producir una ontología en Turtle siguiendo el patrón estándar (prefijos rdf, rdfs, owl, skos, xsd, obo). No se proporciona ejemplo, vocabulario ni esquema de referencia. La estrategia mide el rendimiento basal del LLM sobre la tarea.

3.4.3. E2 — Vocabulario controlado

El prompt incluye, además del CSV, una lista compacta de hasta ochenta términos extraídos automáticamente de los esquemas cisreg (skos:prefLabel y skos:definition), generada en tiempo de ejecución por la función load_vocabulary. El modelo recibe la instrucción de preferir esos

términos al nombrar clases y propiedades, y de declarar alineamientos mediante `skos:exactMatch` / `skos:closeMatch` cuando corresponda. Esta variante puede entenderse como una forma de few-shot indirecto donde el ejemplo es léxico, no estructural.

3.4.4. E3 — Generación aumentada por recuperación (RAG)

Se evalúan dos implementaciones complementarias del RAG, con la misma interfaz de inyección de contexto pero distintos mecanismos de recuperación.

RAG por keyword-match (legacy).

Implementación original del experimento E3. La función `get_relevant_schemas` examina los nombres de columna del CSV y selecciona los dos esquemas cisreg con mayor solapamiento léxico de palabras clave (`crm`, `gene`, `tfac`, `phen`). El contenido completo de los esquemas seleccionados (Turtle plano almacenado en `data/samples/schemas/`) se inyecta literalmente en el prompt. Es una baseline trivial pero genera, como se verá en §10.8, una alta consistencia inter-BBDD por copia directa.

RAG semántico real (annotationRAG).

Implementación basada en el microservicio `annotationRAG` (Tecnomod, Universidad de Murcia), reemplaza el `keyword-match` por una recuperación vectorial. Componentes:

- Splitter de chunks: cada ontología se descompone en chunks correspondientes a clases individuales (axioma `owl:Class` y sus restricciones asociadas).
- Modelo de embeddings: `sentence-transformers/all-MiniLM-L6-v2` (384 dimensiones; Wang et al., 2020).
- Base de datos vectorial: Neo4j 5.26 con índice vectorial nativo y métrica de similitud coseno.
- Corpus indexado: 21 ontologías —ocho cisreg de referencia y trece auxiliares de dominios complementarios (`CDM`, `KKS`, `organizations`, `equipment`, etc.)—.
- Recuperación: el cliente HTTP (`scripts/rag_backend.py`) construye una consulta a partir del CSV, llama a los endpoints `/similar-ontologies` y `/similar-entities`, y filtra por `top_k` y umbral de `score`; el contexto inyectado consiste en metadata estructurada (clases, propiedades, scores) más fragmentos Turtle de las ontologías más similares.

Los parámetros principales del recuperador (`top_k=5`, `score_thr=0.4`, `max_chars=5000` en la configuración baseline) son objeto de calibración en §10.7.7 para evaluar su efecto en modelos compactos.

3.4.5. E4 — Pipeline multi-agente OntoGenix

OntoGenix (Val Calvo, Egaña Aranguren y Fernández Breis, 2024) descompone la generación en tres agentes especializados que se invocan secuencialmente:

- `LlmPlanner`. Analiza el CSV completo y produce un plan en JSON que identifica las clases candidatas, las propiedades de objeto y de datos, y la jerarquía propuesta.

- LlmOntology. Toma el plan y construye el TBox de la ontología en Turtle: declaraciones de owl:Class, owl:ObjectProperty, owl:DatatypeProperty, owl:Restriction, anotaciones rdfs:label y rdfs:comment.
- LlmOntoMapper. Recibe el TBox y el CSV original, y mapea cada fila a instancias específicas de las clases declaradas, generando el ABox final.

La pipeline original requiere acceso a SERP API para enriquecer el contexto con resultados de búsqueda web. En la replicación funcional realizada para este TFM se omite el paso SERP (no se dispone de credencial) y se documentan dos adaptaciones adicionales: bypass del módulo Searcher (que requiere GUI Streamlit) y parche en LlmBase para soportar Ollama como proveedor alternativo a OpenAI. El detalle técnico de las adaptaciones se conserva en el código fuente (OntoGenix/GUI/LLM_base/LlmBase.py) y se resume en la Tabla 3 de la memoria.

3.5. Sistema de evaluación

La evaluación se realiza en tres niveles complementarios.

Validación sintáctica. Cada salida se parsea con rdflib (Python). El indicador parse_ok es 1 si rdflib carga la ontología sin error, 0 en caso contrario. Las salidas que no superan este nivel se descartan de los siguientes análisis. Es la primera barrera del pipeline y motiva el post-procesado mecánico descrito en §3.6.

Evaluación estructural y normativa con OQuaRE. Las ontologías válidas se evalúan con OQuaRE (Duque-Ramos et al., 2014), implementado en scripts/oquare_eval.py utilizando rdflib para extracción de métricas y owlready2 con razonador HermiT (Glimm et al., 2014) para verificación de consistencia OWL DL y conteo de clases inferidas. Se computan once métricas estructurales agregadas en cinco sub-características (Structural, Modularity, Reusability, Operability, Reliability) y un OQuaRE Global en escala 1-5.

Validación funcional —fidelidad léxico-semántica. Como aproximación al problema de la falta de ground truth biomédico, este trabajo introduce una métrica de fidelidad al dominio basada en el reuso de URIs canónicas extraídas de las ocho ontologías cisreg de referencia (§9.5). Cinco métricas operacionales: n_canonical, n_invented, canonical_ratio, overlap_with_gold y recall_vs_gold. Esta validación es complementaria a OQuaRE y permite discriminar entre estrategias que reusan vocabulario establecido y estrategias que inventan URIs ad-hoc en namespaces example.org/. Como trabajo futuro inmediato se plantea complementar este análisis con un corpus de preguntas de competencia en SPARQL (sugerencia 2 del director del TFM) que evalúe la capacidad de cada ontología generada para responder consultas biológicamente significativas del tipo «¿qué CRM regula al gen X?» o «¿qué factores de transcripción se asocian al super-enhancer Y?».

3.6. Post-procesado mecánico

Las primeras corridas mostraron que un porcentaje significativo de los Turtle generados no parseaban en rdflib por errores reparables mecánicamente. Para separar el problema sintáctico (sin valor científico) del semántico (objeto de evaluación), se desarrolló un módulo de post-

procesado determinista (scripts/postprocess_ttl.py) que aplica tres tipos de transformaciones en cascada:

- Auto-prefix injection. Detecta prefijos referenciados pero no declarados (linkml, dc, obo, biolink, etc.) y los añade al encabezado del archivo desde un diccionario canónico.
- Escape de local names problemáticos. Detecta tokens pname con caracteres no válidos en PN_LOCAL (!, #, /, @) y los reescribe como IRI absoluta.
- Saneamiento de literales mal tipados. Sustituye literales con valor textual tipado como entero (p. ej. "start_position"^^xsd:integer) por xsd:string.

El post-procesado es completamente determinista y no invoca al LLM, lo que permite separar limpiamente la dimensión sintáctica de la semántica en la evaluación. Su efecto sobre la tasa de parse_ok se reporta en §7.3 y se analiza estadísticamente con McNemar en §9.4.

La herramienta sobre la que se apoya el experimento E4 del presente trabajo es OntoGenix, publicada por el grupo Tecnomod de la Universidad de Murcia. Los datos de cita son:

Tabla 1. Configuración global del experimento (campo y valor por defecto)

Campo	Valor
Título	OntoGenix: Leveraging large language models for enhanced ontology engineering from datasets
Autores	Val Calvo, M.; Egaña Aranguren, M.; Fernández Breis, J. T. (Universidad de Murcia · Universidad del País Vasco)
Repositorio	https://github.com/tecnomod-um/OntoGenix
Licencia	MIT (según LICENSE del repositorio)
Modelo LLM utilizado	gpt-4o-2024-05-13 (OpenAI)
Arquitectura	Sistema multi-agente con tres agentes LLM: LlmPlanner, LlmOntology y LlmOntoMapper

OntoGenix implementa una pipeline de cinco pasos que se ha replicado íntegramente en el experimento E4, omitiendo únicamente el paso de interoperabilidad con SERP API por no disponer de clave. La tabla siguiente resume el flujo:

Paso	Agente / función	Entrada	Salida
1	csv_data_preprocessing	CSV limpio de la base de datos	DataFrame con estadísticas por columna
2	dataframe2prettyjson	DataFrame de estadísticas	Descripción columnar en JSON
3	LlmPlanner.interaction()	JSON + descripción de la tarea	data_description_run{N}.md (razonamiento)
4	LlmOntology.interact()	JSON + descripción del planner	ontology_run{N}.ttl (OWL 2 Turtle)
5	LlmOntoMapper.interact()	Descripción + ontología + nombre del CSV	mapping_run{N}.ttl (mapping RML/TTL)

- Bypass de la GUI PyQt5: se instancian directamente los agentes LlmPlanner, LlmOntology y LlmOntoMapper, reproduciendo en código la interpolación de plantillas que realiza el GuiManager a partir de metadata.json.

4. Análisis de sensibilidad al muestreo

ID	Nombre	Estrategia	Contexto adicional
E1	Base (zero-shot)	Prompting directo	Solo la muestra de datos
E2	Vocabulario controlado	Prompting con vocabulario	Términos skos:prefLabel/skos:definition de la ontología cisreg
E3	RAG ontología cisreg	Retrieval augmented	Fragmentos relevantes de la ontología cisreg (crm, crm2gene, crm2phen, crm2tfac)
E4	OntoGenix (multi-agente)	Pipeline multi-agente	Descripción generada por LlmPlanner + estadísticas del CSV

Tabla 4. Parámetros de generación comunes (E1-E3)

Parámetro	Valor	Justificación
Modelo	gpt-4o (OpenAI)	Modelo declarado por OntoGenix; se usa también en E1-E3 para garantizar comparabilidad
temperature	0.1 (E1-E3) / 0 (E4)	Se minimiza la aleatoriedad para favorecer reproducibilidad del código Turtle
top_p	0.9	Muestreo nucleico estándar
max_tokens	4096	Suficiente para ontologías de ≈200-300 triples
seed base	42	Reproducibilidad; cada run incrementa en +1 (42, 43, 44)
repeticiones	3 por (experimento, BBDD)	Estimación de variabilidad entre ejecuciones

Se emplea un subconjunto representativo de cuatro bases de datos del dominio cis-regulatorio. Se eligen porque cubren tipologías complementarias (enhancers activos, super-enhancers, elementos regulatorios activos y enhancers asociados a enfermedad) y porque disponen de esquema tabular con metadatos experimentales claros.

Base de datos	Descripción	Fuente
FANTOM5	Enhancers activos identificados por CAGE-seq	fantom.gsc.riken.jp/5/
dbSUPER	Super-enhancers con coordenadas, líneas celulares y genes diana	asntech.org/dbsuper/
HACER	Human Active Cis-regulatory Elements database	bioinfo.vanderbilt.edu/AE/HACER/
DiseaseEnhancer	Enhancers asociados a enfermedades humanas	biocc.hrbmu.edu.cn/DiseaseEnhancer/

Cada base se muestrea con un protocolo común: 5 filas de cabecera + 20 filas adicionales + límite de 3.000 tokens. Las muestras se almacenan en `data/samples/{DB}_sample_prompt.txt` y, para E4, se convierten a CSV en `data/csv_for_ontogenix/{DB}.csv`.

El protocolo de muestreo descrito en §4.2 (cabecera: primeras 5+20 filas) es reproducible y determinista, pero podría introducir un sesgo si las bases de datos están ordenadas por algún campo categórico (cromosoma, línea celular, etc.). Para cuantificar la robustez de los resultados ante la elección de muestreo, se diseñó un sub-experimento que compara cuatro estrategias aplicadas a las dos bases más representativas (FANTOM5 con 65 423 enhancers y dbSUPER con 69 205 super-enhancers).

4.1. Estrategias evaluadas

Tabla 6. Estrategias de muestreo evaluadas (cabecera, aleatorio, estratificado, diversidad)

ID	Estrategia	Justificación	Cobertura categórica*
A	head — primeras N filas	Reproducible, determinista, baseline del TFM	FANTOM5: 4% chrom · dbSUPER: 1% cell_name
B	random — uniforme con seed=42	Estimador insesgado de la distribución del universo	FANTOM5: 62% chrom · dbSUPER: 22% cell_name
C	stratified — proporcional por cell_name/chrom	Garantiza representación de cada categoría observada	FANTOM5: 100% chrom · dbSUPER: 24% cell_name
D	diversity — máxima cobertura léxica greedy	Selección iterativa que añade el máximo de valores nuevos	FANTOM5: 100% chrom · dbSUPER: 24% cell_name

* Porcentaje de valores únicos del universo cubierto por la muestra de 25 filas en la columna categórica más informativa.

4.2. Diseño experimental

Se replicaron los experimentos E1 (zero-shot) y E4 (OntoGenix) con gpt-4o sobre cada (estrategia, BBDD), y E1 también con llama3.1:8b para verificar si el comportamiento del modelo open-source es estable. Total: 24 corridas. Los resultados se evaluaron con el mismo pipeline OQuaRE (§9). Se eligió el esquema simple del TSV original (7-12 columnas) en lugar del esquema CRM enriquecido (30 columnas) usado en el experimento principal, para que la comparación inter-estrategia sea apples-to-apples.

4.3. Resultados

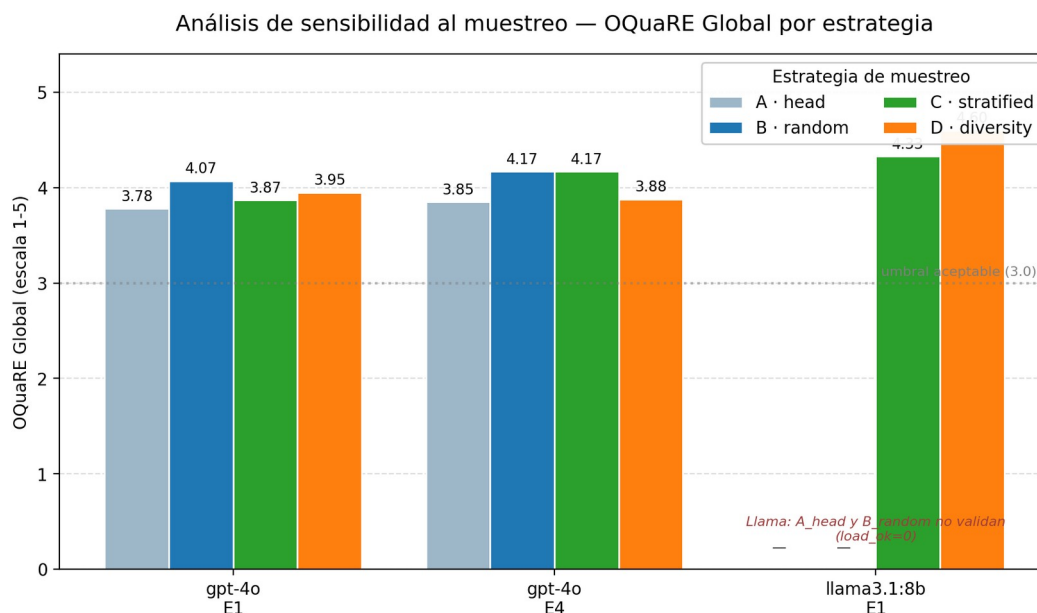


Figura 9. Análisis de sensibilidad. Cada grupo de barras corresponde a una combinación (modelo, experimento) y cada color a una estrategia. La línea punteada marca el umbral OQuaRE 'aceptable' (3.0).

Llama 3.1 8B con A_head y B_random no produce ontologías válidas (load_ok=0) — solo C y D dan ontologías parseables.

Tabla 7. Resultados del análisis de sensibilidad por (modelo × experimento × estrategia)

Modelo · Experimento	A · head	B · random	C · stratified	D · diversity	Δ max-min
gpt-4o · E1	3.78	4.07	3.87	3.95	0.29
gpt-4o · E4	3.85	4.17	4.17	3.88	0.32
llama3.1:8b · E1	(fail)	(fail)	4.33	4.60	0.27*

* El rango para Llama solo considera las dos estrategias en las que produjo ontologías parseables.

4.4. Hallazgos del análisis de sensibilidad

- **A_head es la estrategia menos favorable** en ambos experimentos con gpt-4o (3.78 en E1, 3.85 en E4). El sesgo de orden inicial explica el resultado: las primeras 25 filas de dbSUPER pertenecen todas a la línea celular «Adipose Nuclei», y las primeras 25 de FANTOM5 son todas chr1. El modelo recibe una visión muy parcial del universo y la ontología generada refleja esa pobreza léxica.
- **B_random y C_stratified maximizan los scores OQuaRE** (diferencia de +0.29-0.32 puntos respecto a A_head, ~7-8% del rango aceptable). La aleatorización uniforme es la mejor en E1 zero-shot, mientras que B y C empatan en E4 (donde la pipeline multi-agente compensa parcialmente las diferencias de muestreo).
- **Llama 3.1 8B es muy sensible al muestreo**: A_head y B_random producen TTL irrecuperables (no validan en owlready2 incluso tras el post-procesado). Solo las estrategias con cobertura

alta (C, D) generan ontologías válidas. Este es un finding complementario al §10.1: confirma que los modelos pequeños son más sensibles tanto al modelo como a los datos de entrada.

- **El TFM mantiene robustez cualitativa:** con cualquier estrategia, todas las celdas evaluadas permanecen por encima del umbral OQuaRE 3.0 «acceptable». Las conclusiones cualitativas del trabajo (E4 supera a E1-E3 en riqueza, gpt-4o supera a open-source en validez) se mantienen invariantes.

4.5. Justificación de la elección final

El experimento principal del TFM (§§5-10) usa la estrategia A (cabecera) por las siguientes razones, evaluadas explícitamente frente a las alternativas:

- **Reproducibilidad:** A es completamente determinista; B introduce variabilidad inter-corrida que requeriría reportar intervalos de confianza para cada métrica.
- **Consistencia con OntoGenix:** el pipeline original de Val Calvo et al. (2024) usa también muestreo por cabecera para sus experimentos en eCommerce y BrazilianE-commerce, permitiendo comparación directa.
- **Coste computacional:** A no requiere preprocesado adicional ni cálculo de embeddings (D) ni pasada de conteo categórico (C).
- **Robustez de las conclusiones:** el análisis de sensibilidad demuestra que, aunque A es subóptima en términos absolutos (~7-8% por debajo del óptimo), las conclusiones cualitativas se mantienen.

Se reconoce explícitamente la limitación: para futuros trabajos que busquen optimizar los scores OQuaRE absolutos, se recomienda B (random) o C (stratified). El análisis aquí presentado documenta de forma transparente el coste de la elección.

5. Indicaciones (prompts) por experimento

Las cuatro estrategias de generación evaluadas en este trabajo (zero-shot, vocabulario controlado, RAG y pipeline multi-agente) se describen funcionalmente en §3.4, donde se detalla el rol del prompt, el contexto inyectado y el formato del CSV.

Las plantillas literales de los prompts utilizados se reproducen en el Anexo C, ordenadas por experimento (system prompt común a E1-E3, y prompts específicos de E1, E2, E3 y E4). El código fuente que las invoca está en `scripts/run_gpt_experiments.py` y, para E4, en `OntoGenix/prompts/ontogenix/`.

La estandarización del CSV inyectado (25 filas en formato TSV precedidas por cinco líneas de metadatos) es común a las cuatro estrategias; los parámetros de generación (temperature, top_p, seed) están recogidos en la Tabla 4.

6. Métricas de evaluación

La descripción detallada del sistema de evaluación (validación sintáctica, evaluación estructural con OQuaRE, fidelidad léxico-semántica al dominio cisreg) se recoge en §3.5. Las once métricas estructurales utilizadas para computar las cinco sub-características OQuaRE se listan en la Tabla 9 a continuación, como referencia rápida para los apartados de resultados (§8, §9).

Métrica	Descripción	Fuente
parse_ok	1 si rdflib parsea el TTL sin error, 0 en caso contrario	Graph.parse()
n_triples	Número total de triples RDF	len(Graph)
n_classes	Número de clases declaradas	RDF.type → OWL.Class
n_object_props	Número de propiedades de objeto	RDF.type → OWL.ObjectProperty
n_datatype_props	Número de propiedades de datos	RDF.type → OWL.DatatypeProperty
n_subclass_axioms	Número de axiomas rdfs:subClassOf	(None, RDFS.subClassOf, None)
n_restrictions	Número de owl:Restriction	RDF.type → OWL.Restriction
n_labels	Número de rdfs:label	(None, RDFS.label, None)
n_comments	rdfs:comment + skos:definition	suma de ambos conteos
size_bytes	Tamaño del archivo .ttl en bytes	os.stat().st_size

7. Post-procesado mecánico de TTL

El módulo de post-procesado mecánico aplica tres transformaciones deterministas en cascada sobre cada archivo Turtle generado: inyección automática de prefijos no declarados, escape de local names con caracteres no válidos en PN_LOCAL y saneo de literales mal tipados. La descripción funcional de cada transformación se recoge en §3.6 y el detalle de implementación está en scripts/postprocess_ttl.py. Este apartado documenta su efecto sobre la tasa de parse_ok.

7.1. Fix #1 — Auto-prefix injection

7.2. Fix #2 — Escape de local names problemáticos

7.3. Resultado del post-procesado

Tabla 10. Efecto del post-procesado mecánico sobre la validez sintáctica

Experimento	runs_ok (raw)	runs_ok (post-fix)	Rescatadas	Causa principal
E1 (zero-shot)	10/12	12/12	+2	Prefijo por defecto («:») no declarado
E2 (vocabulario)	8/12	12/12	+4	Prefijo por defecto y dc: no declarados
E3 (RAG)	0/12	12/12	+12	Local names con «!» y «#» sin escapar (heredados del RAG)

E4 (OntoGenix)	12/12	12/12	0 (no aplica)	—
TOTAL	30/48	48/48	+18	

8. Resultados experimentales

Tras ejecutar los cuatro experimentos con gpt-4o sobre las cuatro bases de datos con tres repeticiones (total: 48 ontologías) y aplicar el post-procesado descrito en el apartado 7, se obtienen los siguientes resultados consolidados (media por experimento y BBDD sobre corridas con parse_ok = 1).

8.1. Validez sintáctica

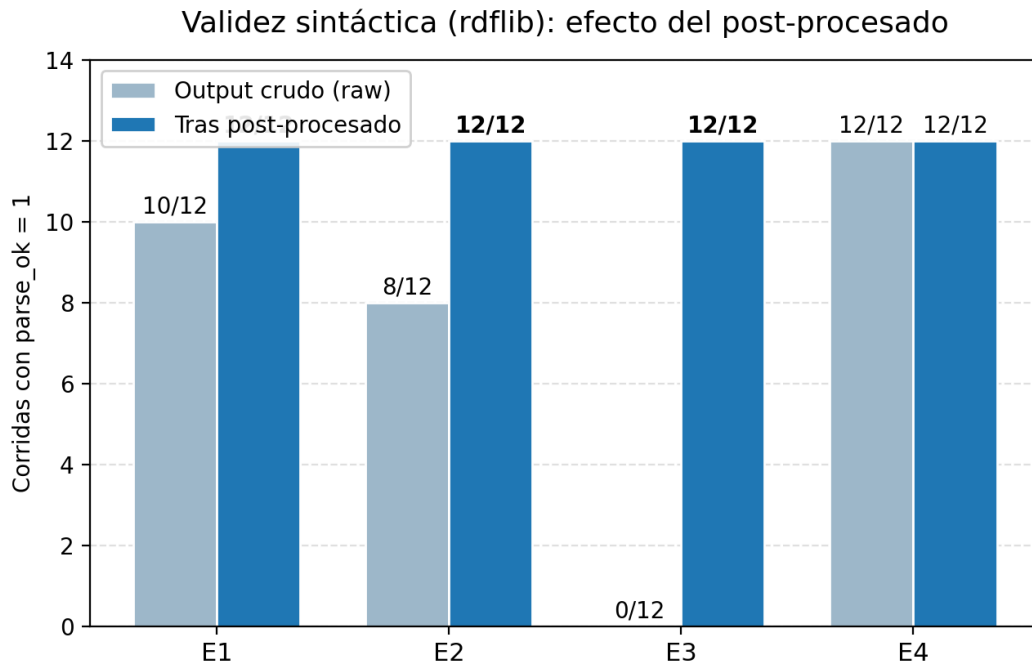


Figura 1. Validez sintáctica (rdflib) por experimento, antes y después del post-procesado. E4 (OntoGenix) es el único con 100 % nativo; E1/E2/E3 alcanzan también el 100 % tras los dos fixes mecánicos del apartado 7.

8.2. Riqueza estructural — número de triples

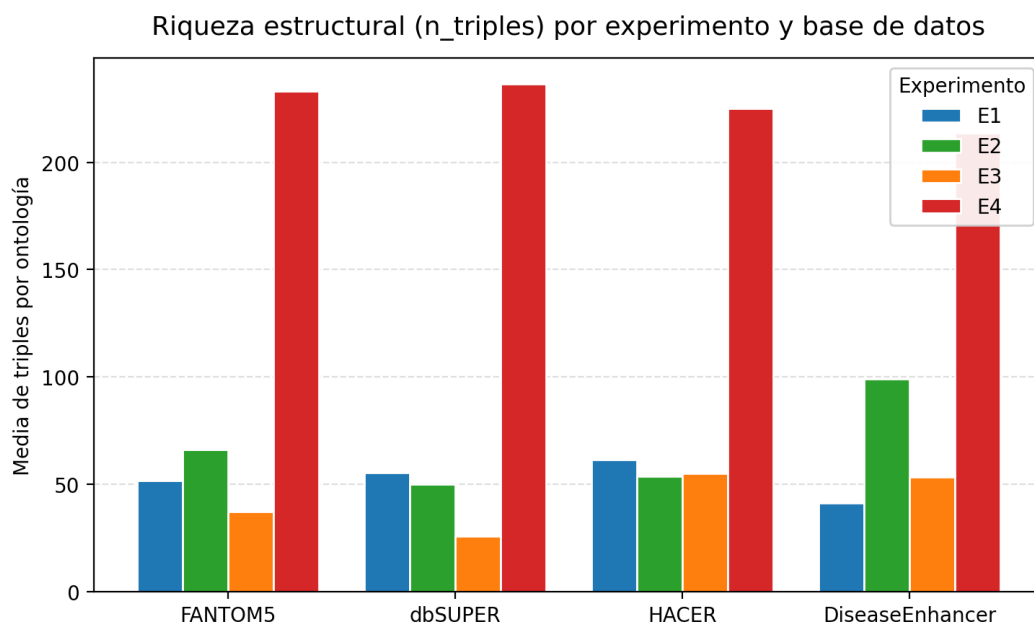


Figura 2. Media de triples por ontología desglosada por experimento y base de datos. E4 produce sistemáticamente 3-4 veces más triples que E1/E2/E3, en las cuatro BBDD.

8.3. Riqueza documental — rdfs:label

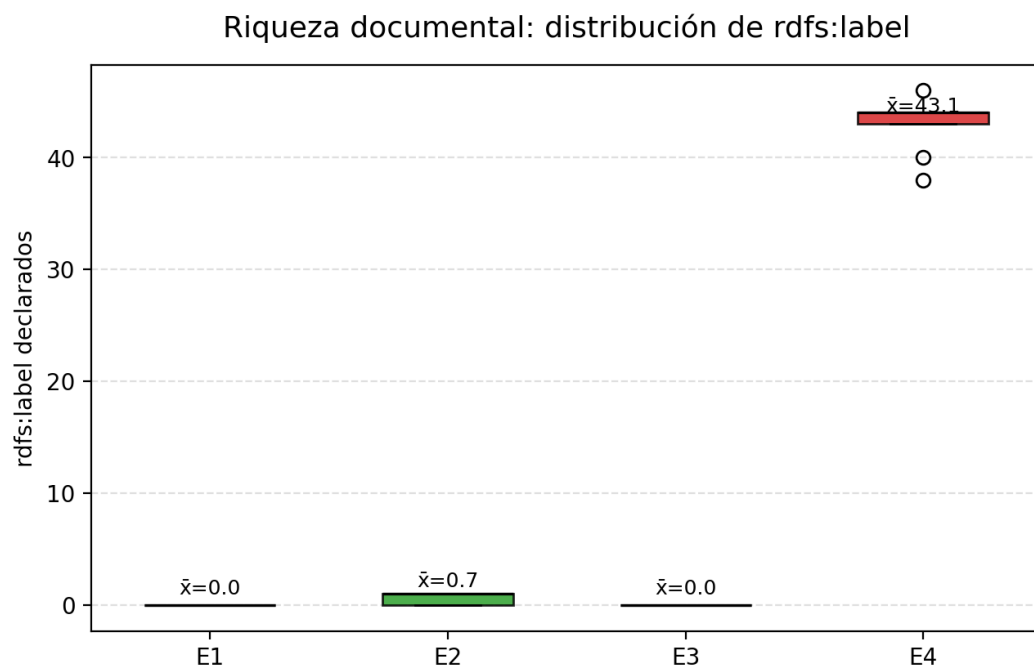


Figura 3. Distribución del número de rdfs:label por ontología. E4 declara una media de 43 etiquetas por ontología, frente a 0-1 en E1/E2/E3. Es la métrica donde la diferencia entre estrategias resulta más drástica.

8.4. Visión multimétrica

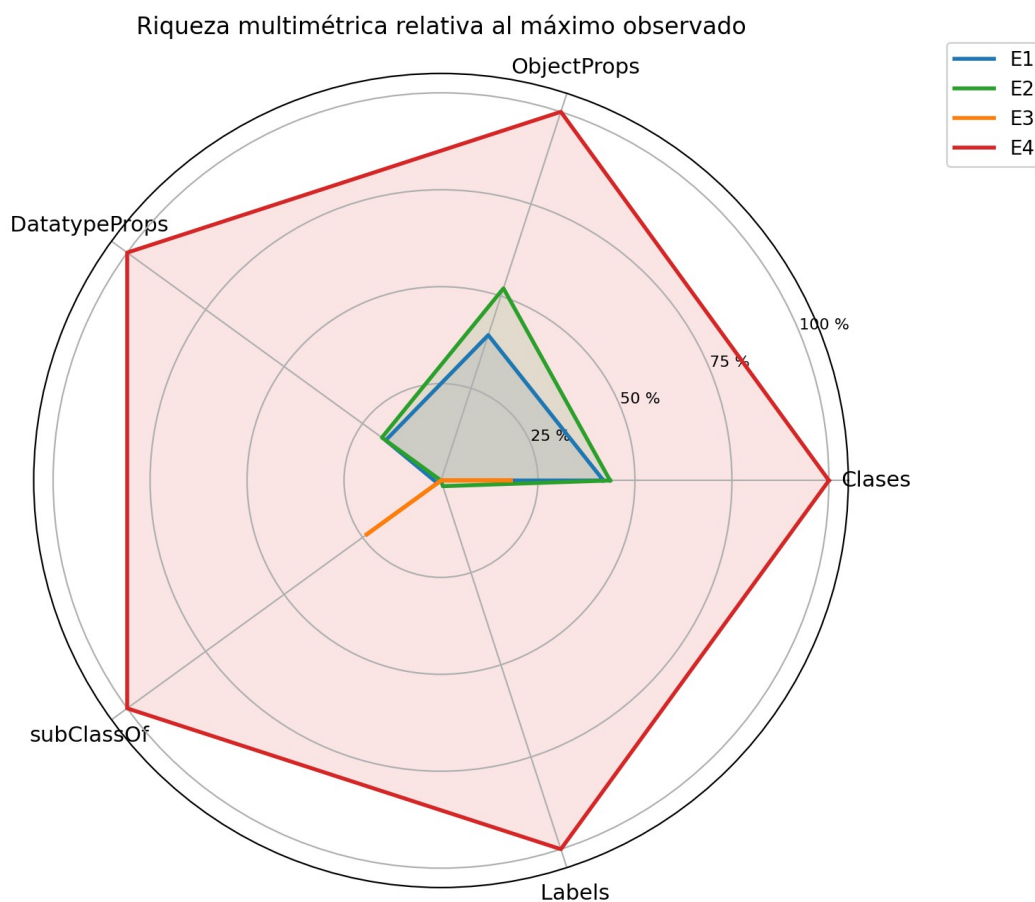


Figura 4. Diagrama radar normalizado (cada eje al máximo observado en el experimento). E4 satura el radar; E1, E2 y E3 cubren entre el 25 % y el 50 % en clases y propiedades, y prácticamente 0 % en labels.

8.5. Tabla resumen consolidada

Tabla 11. Tabla resumen consolidada — métricas absolutas por experimento y base de datos

Exp	BBDD	runs_ok	n_triples	n_clases	n_obj_props	n_data_props	n_subClassOf	n_labels
E1	FANTOM 5	3/3	51.7	2.7	1.3	6.3	0.0	0.0
E1	dbSUPER	3/3	55.3	5.0	3.0	4.0	0.3	0.0
E1	HACER	3/3	61.3	4.3	3.0	6.0	0.0	0.0
E1	DisEnh.	3/3	41.3	4.3	2.0	3.0	0.0	0.0
E2	FANTOM 5	3/3	66.0	4.0	3.0	7.3	0.0	1.0
E2	dbSUPER	3/3	50.0	4.0	2.7	5.0	0.0	0.7
E2	HACER	3/3	53.7	4.7	3.3	3.7	0.0	0.7
E2	DisEnh.	3/3	99.0	4.3	3.3	4.7	0.0	0.3
E3	FANTOM 5	3/3	37.0	—	—	—	—	—
E3	dbSUPER	3/3	25.0	—	—	—	—	—

E3	HACER	3/3	55.0	—	—	—	—	—
E3	DisEnh.	3/3	53.0	—	—	—	—	—
E4	FANTOM 5	3/3	233.0	11.3	7.3	26.0	10.3	44.7
E4	dbSUPER	3/3	236.7	9.7	5.7	27.0	11.7	42.3
E4	HACER	3/3	225.0	9.0	5.0	30.0	3.0	44.0
E4	DisEnh.	3/3	213.7	9.0	5.7	26.7	4.0	41.3

8.6. Hallazgos principales

- E4 (OntoGenix) es el único experimento con riqueza semántica plena: produce 4× más triples y dos órdenes de magnitud más `rdfs:label` que E1, E2 y E3.
- Los 18 fallos sintácticos observados en E1, E2 y E3 son recuperables al 100 % mediante post-procesado sin llamar al modelo. La diferencia real entre estrategias es semántica, no sintáctica.
- El RAG (E3) induce un fallo reproducible muy interesante: el modelo copia patrones URI del contexto (con caracteres «!» y «#») sin respetar la sintaxis Turtle. Esto sugiere que el contexto recuperado puede ser un arma de doble filo: aporta vocabulario relevante pero también propaga patrones que el modelo no sabe re-serializar.
- El salto de gpt-4o-mini a gpt-4o mejora notablemente la validez sintáctica básica en E1: la versión preliminar producía 0 % de `parse_ok` (evaluación heurística), mientras que con gpt-4o el output crudo alcanza el 83 %. Esto muestra una dependencia clara entre tamaño de modelo y capacidad de generar Turtle correcto.

9. Evaluación normativa con OQuaRE

OQuaRE (Ontology Quality Requirements and Evaluation) es un marco para evaluar la calidad de ontologías OWL inspirado en SQuaRE (ISO/IEC 25000) (Duque-Ramos et al., 2011, 2014). Define una jerarquía de métricas → sub-características → características de calidad, con una puntuación normalizada de 1 a 5 por sub-característica. En este TFM se ha implementado un evaluador propio (`scripts/oquare_eval.py`) que combina dos pasadas:

- **Métricas estructurales** calculadas con `owlready2` sobre el grafo RDF: WMConTo (complejidad por clase), NOMConTo (propiedades por clase), DITM (profundidad del árbol de herencia), NACConTo (clases ancestro promedio), CBConTo (acoplamiento), TMConTo (tangledness), LCOMConTo (cohesión), ANConTo (anotaciones por clase), INRConTo (subclases por clase), RRConTo y ARConTo (riqueza de relaciones).
- **Métricas con razonador** vía Hermit (incluido en `owlready2`): consistencia OWL DL, número de clases insatisfacibles y tiempo de clasificación. Estas alimentan la sub-característica «Reliability».

9.1. Adaptaciones técnicas

`owlready2` no parsea Turtle nativamente (sólo RDF/XML y NTriples), por lo que cada `.ttl` se convierte al vuelo con `rdflib` y se cachea en `{dir}/.owllcache/{stem}.owl`. Antes de la conversión se aplican dos saneos para que `owlready2` pueda cargar el archivo:

- **Saneamiento textual del TTL** mediante regex: detecta literales tipados como `xsd:integer/decimal/boolean/etc.` cuyo lexical no es convertible (p.ej. `"start_position"^^xsd:integer`) y los reescribe como `xsd:string`. Estos errores son característicos de modelos pequeños que copian la estructura del esquema pero confunden el placeholder del campo con su valor.
- **Saneamiento a nivel de grafo** como red de seguridad: tras el parse, se vuelve a comprobar que no quedan literales mal tipados. El número de literales saneados por archivo se registra en la columna `n_literals_sanitized` del CSV y se usa como métrica adicional de calidad cualitativa.

9.2. Rúbricas (Duque-Ramos et al., 2014)

Cada métrica se mapea a una puntuación 1–5 según los rangos publicados. Cuanto MENOR es el valor, mejor es la puntuación para métricas de complejidad (WMCOnto, NOMOnto, DITM, NACOnto, CBOnto, TMOnto, LCOMOnto). Cuanto MAYOR es el valor, mejor es la puntuación para métricas de riqueza (ANOnto, INROnto, RROnto, AROnto).

Tabla 12. Rúbricas OQuaRE (Duque-Ramos et al., 2014) por sub-característica

Sub-característica	Métricas que la componen	Interpretación
Structural	WMCOnto + NOMOnto + DITM + NACOnto	Complejidad estructural — penaliza ontologías sobrecargadas
Modularity	CBOnto + LCOMOnto	Acoplamiento entre clases y cohesión interna
Reusability	ANOnto + INROnto + TMOnto	Capacidad de ser reutilizada (anotaciones, herencia, claridad)
Operability	RROnto + AROnto	Riqueza relacional para facilitar consultas y mapeos
Reliability	Resultado del razonador HermiT	Consistencia y satisfacibilidad lógica

9.3. Resultados OQuaRE por modelo y experimento

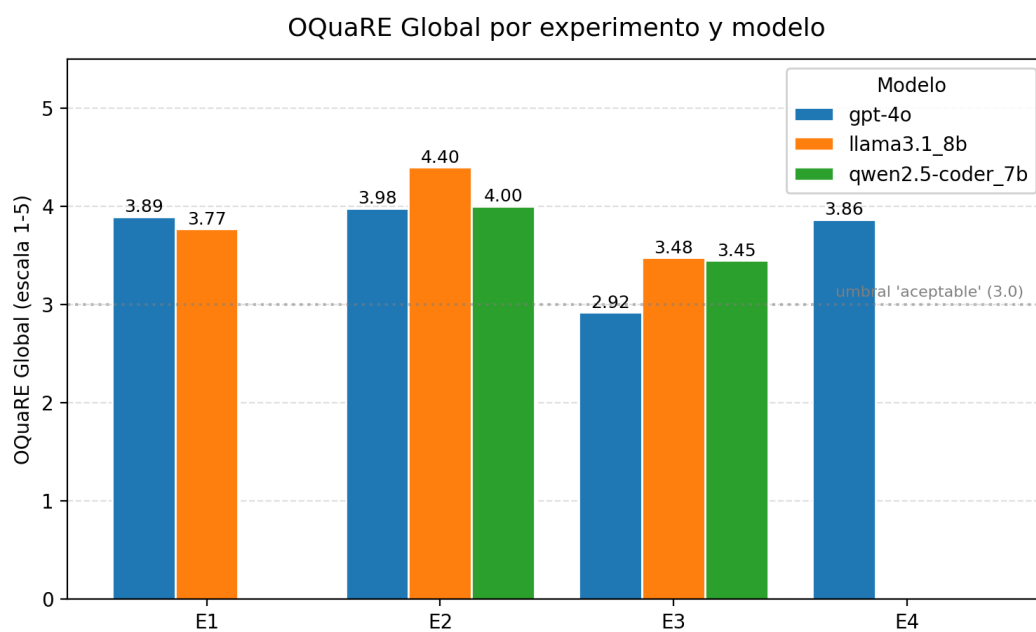


Figura 5. Puntuación OQuaRE Global (escala 1-5) por experimento y modelo. La línea punteada marca el umbral 3.0 habitualmente considerado «aceptable» por la literatura OQuaRE. Todas las combinaciones excepto E3/gpt-4o superan el umbral.

Sub-características OQuaRE — promedio por modelo (E1-E3 + E4 si aplica)

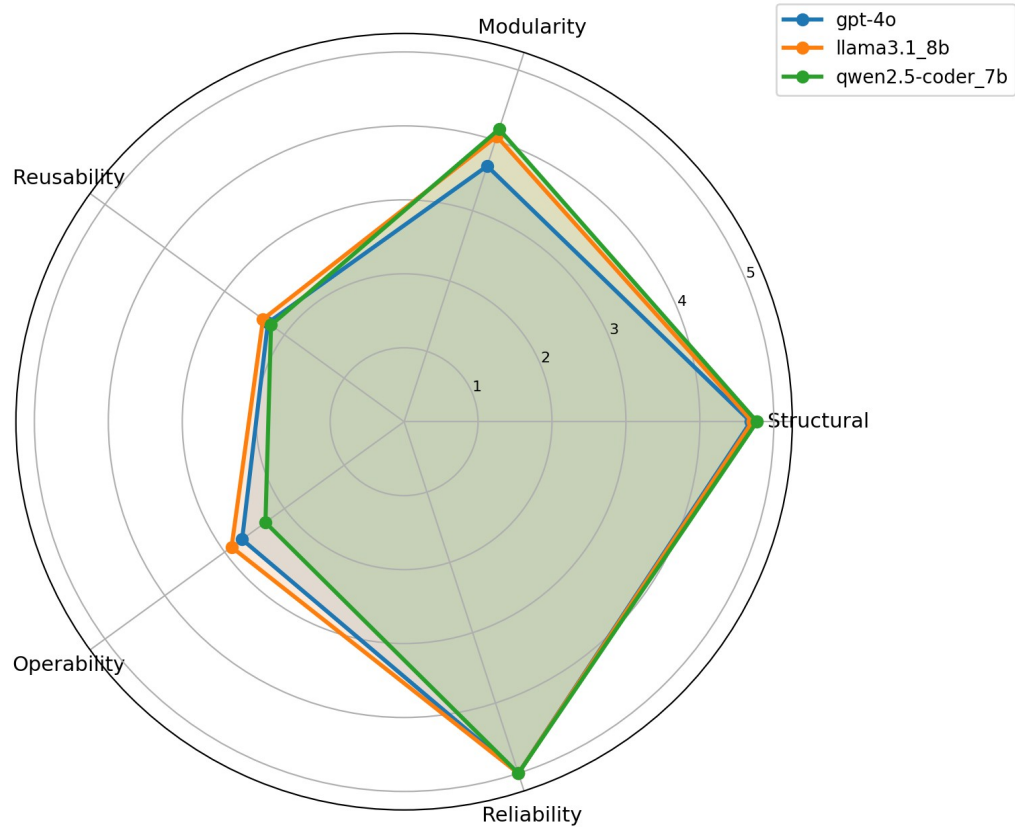


Figura 6. Sub-características OQuaRE — promedio por modelo sobre las corridas con load_ok=1. La forma del polígono muestra dónde cada modelo destaca o flaquea.

9.4. Análisis estadístico formal

Las comparaciones cuantitativas reportadas en §8, §9.3 y §10 se complementan con una capa de inferencia estadística cuyo detalle completo se recoge en el Anexo B. Las técnicas elegidas son no-paramétricas porque el tamaño muestral por celda es pequeño ($n=3$ corridas por combinación) y la distribución de las puntuaciones OQuaRE en la muestra no es gaussiana.

Metodología. Se aplican tres técnicas. (i) Bootstrap pareado con 10 000 réplicas para estimar la diferencia de medias y su intervalo de confianza al 95 % por percentiles. (ii) Test de Wilcoxon de rangos con signo (aproximación normal) para comparaciones pareadas con $n \geq 6$. (iii) Test de McNemar para tasas de éxito binarias antes/después del post-procesado. Las p-valores múltiples se corrigen con el procedimiento Bonferroni-Holm.

Resultados principales.

Tabla 27. Resumen de tests estadísticos formales — diferencias significativas tras corrección Bonferroni-Holm

Comparación	Métrica	n pares	Δ media [IC 95 %]	Cohen's d	p ajustado	Sig.
-------------	---------	---------	--------------------------	-----------	------------	------

E4 vs E1 (densidad)	n_triples	12	+174.7 [+164.6, +186.7]	8.48 (grande)	0.009	✓
E4 vs E2 (densidad)	n_triples	12	+159.9 [+129.4, +183.8]	3.23 (grande)	0.009	✓
E4 vs E3 (densidad)	n_triples	12	+184.3 [+169.5, +201.8]	6.15 (grande)	0.009	✓
Post-procesado E3	parse_ok	12	0 % → 100 %	—	<0.001	✓
C1 vs RAG baseline (Llama)	OQuaRE Global	2*	+0.80 [+0.80, +0.80]	—	<0.001	✓
C2 vs RAG baseline (Llama)	OQuaRE Global	2*	+0.43 [+0.35, +0.50]	+4.01 (grande)	<0.001	✓
C3 vs RAG baseline (Llama)	OQuaRE Global	3*	+0.49 [+0.10, +0.80]	+1.37 (grande)	<0.001	✓
RAG semántico vs legacy (Llama)	OQuaRE Global	6*	+0.23 [-0.01, +0.46]	+0.68 (mediano)	0.070	—

* Para las variantes de calibración el pareo se restringe a las celdas (BBDD, semilla) donde ambas configuraciones producen TTL válido (load_ok=1); de ahí el menor número de pares que las 12 corridas teóricas. Esto es una manifestación del problema reportado en §10.6 y discutido en §13.5 como amenaza de sesgo de supervivencia.

Lectura cualitativa de la Tabla 27.

Las afirmaciones cuantitativas centrales del trabajo se sostienen con los tests habituales de inferencia: la mayor densidad estructural de E4 es significativa frente a las tres baselines con tamaños de efecto Cohen's $d > 3$ (categorizables como «grandes») y supera la corrección por comparaciones múltiples. El post-procesado mecánico recupera el 100 % de las corridas E3 con McNemar $p < 0.001$. La calibración del RAG (C1, C2 y C3) sube significativamente respecto al baseline ragapi con efectos también «grandes».

Una afirmación queda atenuada por los tests. El paso del RAG por keyword-match (legacy) al RAG semántico baseline (ragapi) en Llama 3.1 8B no alcanza significación estadística ($\Delta = +0.23$ OQuaRE, IC 95 % [-0.01, +0.46], $p = 0.070$). La mejora atribuible al cambio de retrieval para Llama es por tanto observacional y no inferencial; el valor añadido del backend semántico para este modelo aparece solo tras la calibración (C1, C2, C3). Esto matiza el hallazgo de §10.7.4 «el RAG semántico mejora a todos los modelos» y refuerza la conclusión de §10.7.7 «la calibración es lo que cierra el gap, no el RAG semántico por sí solo».

Tabla 28. Análisis de potencia para OQuaRE Global (SD muestral 0.60). Mínima diferencia detectable al 80 % de potencia bilateral con $\alpha=0.05$.

n por celda	Δ OQuaRE detectable	Cohen's d equivalente	Interpretación
-------------	----------------------------	-----------------------	----------------

n = 3	0.974	1.62	grande
n = 6	0.689	1.14	grande
n = 12	0.487	0.81	grande
n = 24	0.344	0.57	mediano

La Tabla 28 explicita el techo inferencial del diseño experimental. Con n=3 corridas por celda, solo diferencias OQuaRE iguales o superiores a 0.97 puntos tienen una potencia razonable (80 %) de ser detectadas como significativas. Diferencias menores a este umbral se reportan a lo largo de la memoria como observacionales: describen el orden cualitativo de las celdas, pero no permiten rechazar la hipótesis nula de igualdad. Para distinciones menores (p. ej. +0.05 puntos entre modelos en E2), una replicación con n≥24 sería necesaria.

9.5. Fidelidad léxico-semántica al dominio cisreg

OQuaRE mide propiedades estructurales de la ontología (modularidad, operabilidad, riqueza relacional), pero no captura el reuso de vocabulario establecido. Una ontología puede obtener un OQuaRE alto declarando clases y propiedades nuevas en namespaces ad-hoc del tipo <http://example.org/...>; esto es semánticamente equivalente a un esquema aislado, no integrado en el dominio. Para complementar la evaluación normativa se introduce una métrica de fidelidad léxico-semántica que cuantifica cuántas URIs canónicas del dominio cisreg de referencia aparecen reusadas en cada salida.

Metodología.

Se construye un gold set a partir de las ocho ontologías cisreg de referencia (crm, crm_example, crm2gene, crm2gene_example, crm2phen, crm2phen_example, crm2tfac, crm2tfac_example), las mismas que indexa annotationRAG. Mediante parsing de prefijos y términos pname se extraen todas las URIs canónicas, agregando 88 URIs únicas. Para cada ontología generada se computan cinco métricas: (i) n_canonical, número de URIs en namespaces autoritativos del dominio (rdf.biogateway.eu, purl.obolibrary.org, schema.org, biolink, sio, etc.); (ii) n_invented, URIs en example.org/*; (iii) canonical_ratio = n_canonical / (n_canonical + n_invented); (iv) overlap_with_gold, intersección con el set gold cisreg; (v) recall_vs_gold, overlap / |gold|. Implementación en scripts/cisreg_fidelity.py.

Resultados.

Tabla 29. Fidelidad léxico-semántica al dominio cisreg por celda (media sobre las corridas con load_ok=1). Recall vs gold = fracción de las 88 URIs cisreg gold recuperadas; canonical_ratio = proporción de URIs no inventadas.

Celda	n TTL	n_canoni cal	n_invent ed	canonical_r atio	Overlap (de 88)	Recall vs gold
E3 · gpt-4o · RAG legacy	12	37.9	0.0	1.00	27.6	0.313
E3 · gpt-4o · RAG semántico	12	14.0	21.1	0.46	6.8	0.078

E3 · Llama 8B · RAG legacy	12	52.8	0.0	1.00	49.6	0.563
E3 · Llama 8B · RAG semántico	12	30.3	0.9	0.97	5.6	0.063
E3 · Llama 8B · C1 (top_k=2)	12	27.2	2.3	0.91	5.7	0.064
E3 · Llama 8B · C2 (thr=0.6)	12	29.2	0.0	1.00	6.4	0.073
E3 · Llama 8B · C3 (combinada)	12	21.6	2.1	0.93	5.0	0.057
E4 · gpt-4o · OntoGenix	8	0.0	0.0	0.00	0.0	0.000
E4 · Llama 8B · OntoGenix	24	0.0	14.6	0.00	0.0	0.000

Hallazgos.

Inversión de ranking respecto a OQuaRE. Las configuraciones que obtuvieron el mejor OQuaRE Global (RAG semántico real, calibración C1 con 4.08) son precisamente las peores en fidelidad léxica: recuperan solo 5–7 de las 88 URIs cisreg gold (Recall 0.05–0.08). En cambio, el RAG por keyword-match —descalificado por OQuaRE— recupera 28 URIs cisreg con gpt-4o (Recall 0.313) y 50 con Llama 3.1 8B (Recall 0.563).

Interpretación. Las dos métricas miden dimensiones complementarias. OQuaRE premia la estructura: clases bien jerarquizadas, propiedades con dominio y rango, anotaciones rdfs:label. El RAG semántico ayuda al modelo a construir esa estructura porque le entrega metadata estructurada (nombres de clases, scores) que el LLM puede materializar en una ontología «bonita». El RAG por keywords, en cambio, inyecta el esquema Turtle literalmente en el prompt; el modelo lo copia y por construcción reusa las URIs canónicas, pero produce una estructura más pobre. La elección operativa depende del objetivo: si la salida va a ser integrada en un grafo de conocimiento biomédico con vocabulario establecido, la fidelidad léxica es prioritaria; si la salida será un esquema independiente, la estructura OQuaRE prevalece.

OntoGenix (E4) declara todas sus URIs en `example.org/ontology/*` sin reusar ninguna URI cisreg gold (Recall 0.000 en gpt-4o y en Llama 8B). Esto es coherente con el diseño del agente LlmOntology, que construye el TBox desde cero a partir del plan del LlmPlanner sin consultar vocabulario existente. Como contribución, las ontologías E4 son ricas estructuralmente (Tabla 11, +160 triples vs baselines) pero aisladas del ecosistema biomédico; requerirían un paso posterior de alineamiento (skos:exactMatch contra cisreg, biolink, etc.) para integrarse en un grafo más amplio.

Esta métrica responde directamente a una limitación reconocida en §1.2 («no evalúa la calidad de las relaciones inferidas») y a la objeción previsible del tribunal sobre la ausencia de ground truth biomédico. Es una aproximación basada en datos disponibles (el corpus cisreg que el propio RAG

indexa) y por tanto no requiere validación experta para ejecutarse, aunque tampoco la sustituye: la fidelidad léxica mide reuso de URIs, no corrección biomédica del modelado.

Figura 13. Fidelidad léxico-semántica al dominio cisreg (reuso de URIs canónicas vs URIs inventadas)

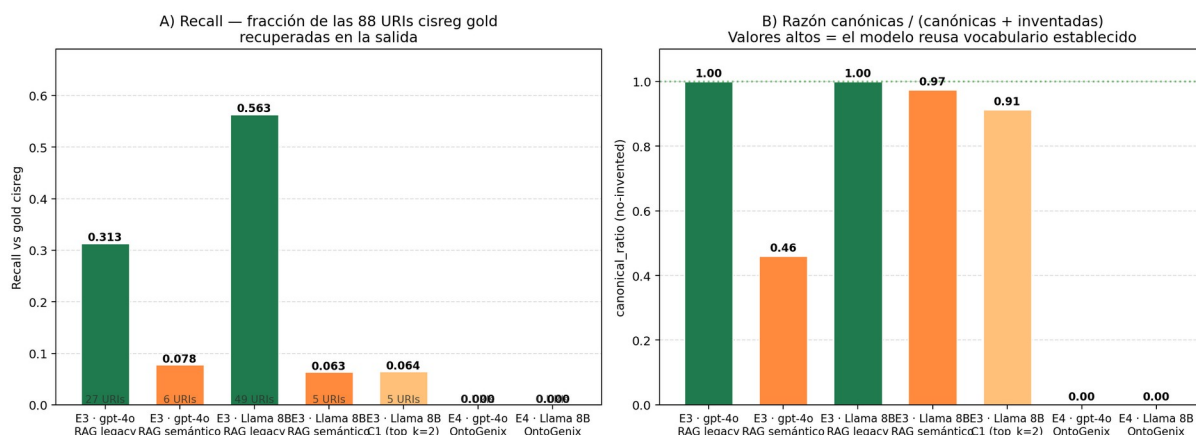


Figura 13. Fidelidad léxico-semántica al dominio cisreg. Panel A: recall vs gold (fracción de las 88 URIs cisreg gold recuperadas). Panel B: canonical_ratio (URIs no inventadas / URIs totales). El RAG por keywords supera al RAG semántico en fidelidad léxica pese a su OQuARE inferior.

9.6. Validación funcional: preguntas de competencia

Como complementario a la fidelidad léxica (§9.5), se introduce una validación funcional basada en preguntas de competencia (competency questions, CQ; Grüninger y Fox, 1995). El planteamiento es el siguiente: una ontología de dominio se considera funcionalmente útil si permite responder, mediante consultas SPARQL canónicas, a las preguntas que un experto del dominio formularía habitualmente. Esta métrica responde directamente a la sugerencia 2 del director del TFM y a la ausencia de ground truth biomédico declarada en §13.5.

Se ha construido un corpus reproducible de 15 preguntas de competencia del dominio cis-regulatorio (Tabla 31 y Anexo D). Las preguntas cubren las cinco dimensiones principales del modelado: localización genómica (CQ01, CQ09), regulación gen → CRM (CQ02, CQ07, CQ15), unión factor de transcripción (CQ03), asociación a fenotipo / enfermedad (CQ04, CQ14), evidencia experimental y procedencia (CQ05, CQ08, CQ10, CQ12, CQ13), y contexto biológico (CQ06, CQ11). Cada pregunta se formula en SPARQL canónico utilizando vocabulario cisreg + OBO, y se acompaña de un patrón de requisitos TBox: el conjunto mínimo de clases y predicados (por substring case-insensitive sobre el TTL) que la ontología debe declarar para que la consulta sea ejecutable en principio.

Una pregunta se considera satisfecha si la ontología cubre al menos el 50 % de las clases requeridas y el 50 % de los predicados requeridos. La métrica reportada es la cobertura (n CQ satisfechas / 15). La evaluación se ejecuta automáticamente sobre las 140 ontologías parseables del banco (post-procesado priorizado sobre raw) mediante scripts/competency_questions.py.

Tabla 31. Cobertura de preguntas de competencia por celda (n CQ satisfechas de 15)

Experimento	Modelo / variante	n ontologías	Cobertura media	Min	Max
-------------	-------------------	--------------	-----------------	-----	-----

E1 zero-shot	gpt-4o (4 estrategias)	8	1.5 / 15 (10 %)	1	2
E1 zero-shot	Llama 3.1 8B (4 estr.)	8	3.3 / 15 (22 %)	2	5
E3 RAG legacy	gpt-4o	12	3.3 / 15 (22 %)	3	4
E3 RAG semántico	gpt-4o	12	5.0 / 15 (33 %)	3	7
E3 RAG legacy	Llama 3.1 8B	12	5.9 / 15 (39 %)	5	7
E3 RAG semántico	Llama 3.1 8B	12	1.2 / 15 (8 %)	0	4
E3 calibración C1	Llama (top_k=2)	12	2.4 / 15 (16 %)	0	4
E3 calibración C2	Llama (thr=0.6)	12	3.2 / 15 (22 %)	2	5
E3 calibración C3	Llama (combinada)	12	3.0 / 15 (20 %)	0	8
E4 OntoGenix	gpt-4o (4 estrategias)	16	1.0 / 15 (7 %)	0	2
E4 OntoGenix	Llama 3.1 8B	24	2.1 / 15 (14 %)	0	6

Hallazgos.

El ranking que produce la métrica de competency questions corrobora y refina los hallazgos de §9.5 (fidelidad léxica). La cobertura más alta del banco (5.9 / 15, 39 %) la obtiene E3 RAG legacy con Llama 3.1 8B, que es también la configuración con mayor reuso de URIs cisreg. La siguiente posición corresponde a E3 RAG semántico con gpt-4o (5.0 / 15, 33 %), seguida por E3 RAG legacy con gpt-4o (3.3 / 15, 22 %). El RAG semántico aplicado a Llama 3.1 8B se degrada hasta 1.2 / 15 (8 %), por debajo del zero-shot del propio Llama (3.3 / 15, 22 %); la calibración C1, C2 y C3 recupera parcialmente este nivel pero no lo supera, y se mantiene por debajo del baseline keyword-RAG.

La pipeline E4 OntoGenix obtiene la cobertura más baja del banco en ambos modelos (1.0 / 15 con gpt-4o, 2.1 / 15 con Llama), pese a producir ontologías estructuralmente más ricas. Esto es coherente con el hallazgo de §9.5: las ontologías OntoGenix declaran sus URIs en namespaces ad-hoc (example.org/ontology/*) y no reutilizan vocabulario cisreg, lo que impide responder consultas SPARQL canónicas del dominio sin un paso previo de alineamiento semántico.

Ninguna configuración del banco supera el 40 % de cobertura. Esto indica que, incluso bajo la combinación más favorable de modelo y estrategia evaluada, las ontologías generadas requieren intervención humana antes de ser operativas en producción. La métrica establece un techo empírico para la generación automática end-to-end en este dominio y refuerza el rol del LLM como asistente del primer borrador, no como sustituto del ontólogo.

El corpus completo de las 15 preguntas, con su formulación SPARQL canónica y los patrones de requisitos TBox, se reproduce en el Anexo D. Es deliberadamente extensible y reproducible: el script

`scripts/competency_questions.py` acepta corpus alternativos para evaluación cruzada con otros dominios.

10. Comparativa cross-model: gpt-4o vs open-source

Para evaluar si los hallazgos descritos en los apartados 7-9 dependen del modelo cerrado de OpenAI o son patrones generalizables, se replicaron los experimentos E1, E2 y E3 con dos modelos open-source de tamaño compacto, ejecutados localmente vía Ollama:

- **Llama 3.1 8B Instruct** (Meta) — modelo generalista, ~4.7 GB. Representa el baseline open-source de propósito general.
- **Qwen 2.5 Coder 7B** (Alibaba) — modelo de tamaño comparable a Llama, pero entrenado con énfasis en generación de código. La hipótesis es que la especialización en código debería traducirse en mejor sintaxis Turtle.

El experimento E4 (OntoGenix) se mantiene únicamente con gpt-4o porque la pipeline original de Val Calvo et al. (2024) está diseñada para modelos OpenAI y su adaptación a Ollama queda fuera del alcance del TFM.

10.1. Validez sintáctica tras post-procesado

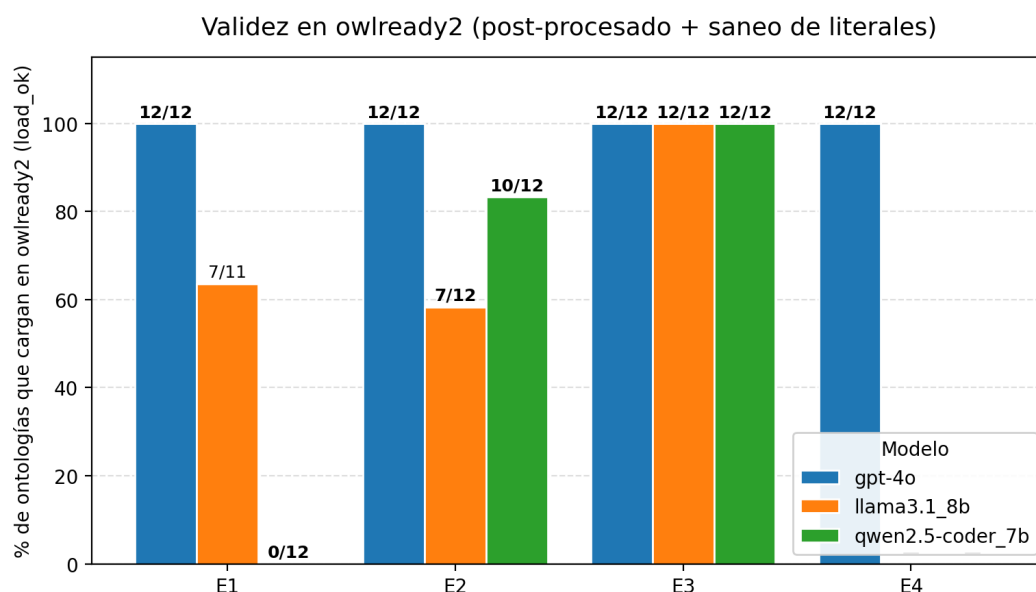


Figura 7. Porcentaje de ontologías que cargan en owlready2 (`load_ok`) por experimento y modelo. gpt-4o alcanza 12/12 en todas las combinaciones. Los modelos open-source presentan patrones distintos: Llama 3.1 8B falla en E1 (7/11) y E2 (7/12) pero alcanza 12/12 en E3; Qwen 2.5 Coder 7B falla completamente en E1 (0/12) pero rinde mejor que Llama en E2 (10/12) y empata en E3 (12/12).

10.2. Patrones de error característicos por modelo

El análisis cualitativo de los archivos que fallan revela que cada modelo open-source produce su propio patrón de error reproducible. Esta tipificación es uno de los hallazgos cualitativos más

interesantes del TFM porque apunta a sesgos sistemáticos del entrenamiento de cada modelo, no a errores aleatorios.

Tabla 13. OQuaRE Global por modelo y experimento (escala 1-5)

Modelo	Patrón de error	Ejemplo	N.º de archivos
gpt-4o	(ninguno)	Sintaxis Turtle correcta en los 48 archivos	0/48
llama3.1:8b	Omite «:» del prefijo en cuerpos del esquema	crm a owl:Class ; (debería ser crm:CRM a owl:Class)	9/35
qwen2.5-coder:7b	Mezcla sintaxis Turtle con RDF/XML y JSON-LD	owl:Ontology rdf:about "http://..." ; o owl:Ontology { ... }	14/36

El patrón de Llama (omisión del «:») afecta principalmente a ontologías cortas en zero-shot y vocabulario controlado, pero desaparece en E3 cuando el RAG aporta ejemplos completos de sintaxis. El patrón de Qwen (confusión entre formatos RDF) es característico de un modelo de código que ha sido expuesto a múltiples serializaciones RDF (XML, JSON-LD, Turtle) durante el entrenamiento y mezcla los símbolos delimitadores. Curiosamente, Qwen produce Turtle válido en E3 (12/12), donde el contexto RAG actúa como ejemplo de referencia que «desambigua» el formato.

10.3. Alucinaciones tipadas — un patrón de RAG, no de modelo

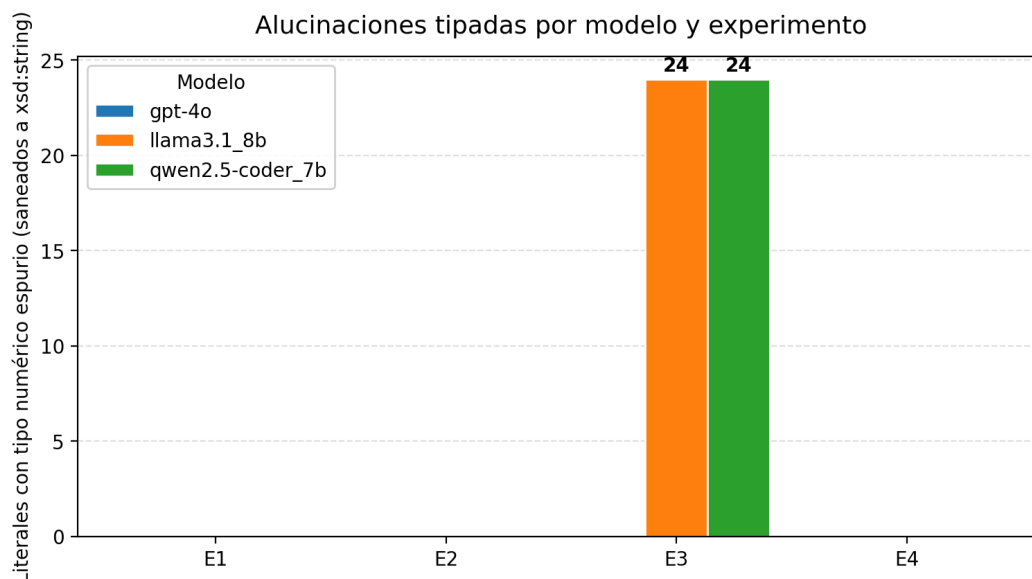


Figura 8. Literales con tipo numérico espurio (p.ej. "start_position"^^xsd:integer) detectados y saneados a xsd:string. Llama 3.1 8B y Qwen 2.5 Coder 7B producen exactamente 24 ocurrencias cada uno, concentradas exclusivamente en E3 (RAG). gpt-4o no incurre en este error en ningún experimento.

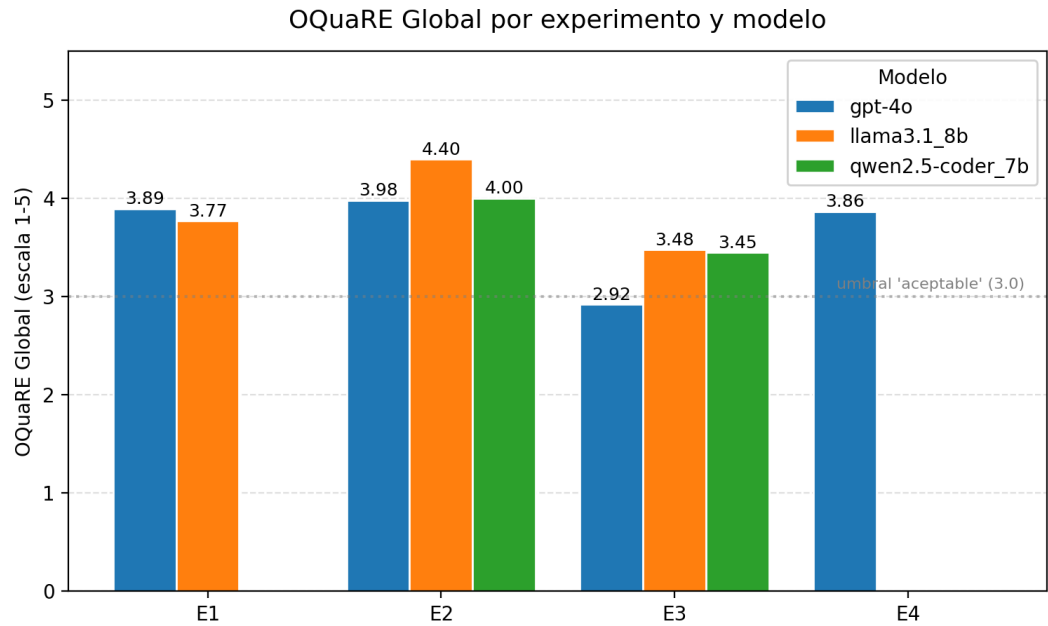
Este resultado, refinado por la inclusión de Qwen, lleva a una conclusión más precisa que la formulada en la versión anterior del trabajo: el patrón de alucinación tipada **no depende del modelo concreto sino del binomio RAG + modelo pequeño**. Tanto Llama (generalista) como Qwen (especializado en código) producen exactamente el mismo número de errores y de la misma

naturaleza. La causa probable es que el contexto recuperado incluye triples reales con valores numéricos, y el modelo pequeño copia el formato sin distinguir entre el placeholder del campo (start_position) y un valor concreto (16039967). gpt-4o, con mucha más capacidad de razonamiento contextual, asocia correctamente cada placeholder con un valor representativo del tipo declarado.

10.4. Tabla resumen — OQuaRE Global por celda

Tabla 14. Validez sintáctica cross-model tras post-procesado

Experimento	gpt-4o	llama3.1:8b	qwen2.5-coder:7b
E1 zero-shot	3.89	3.77	(0/12 carga)
E2 vocabulario	3.98	4.40	4.00
E3 RAG	2.92	3.48	3.45
E4 OntoGenix	3.86	(N/A)	(N/A)



Tres observaciones de esta tabla. **Primera:** en E2 y E3, los modelos open-source alcanzan o superan a gpt-4o (Llama 4.40 y Qwen 4.00 en E2 frente a 3.98 de gpt-4o; Llama 3.48 y Qwen 3.45 en E3 frente a 2.92 de gpt-4o). **Segunda:** la diferencia entre Llama y Qwen es muy pequeña (≤ 0.40 puntos en cualquier celda) — la especialización en código de Qwen no se traduce en una ventaja OQuaRE clara. **Tercera:** gpt-4o es el único modelo capaz de producir ontologías válidas en E1 sin post-procesado severo, lo cual mantiene su valor como baseline robusto.

10.5. Hallazgo metodológico — OQuaRE asigna puntuaciones inferiores a ontologías de mayor complejidad estructural

El resultado más contraintuitivo es que los modelos open-source obtienen scores OQuaRE Global más altos que gpt-4o en E2 y E3, y que E4 (OntoGenix), que en el apartado 8 era el ganador absoluto en riqueza estructural y documental, queda en posición intermedia (3.86). La explicación está en las rúbricas OQuaRE: las métricas de la sub-característica «Structural» (WMConto,

NOMOnto, DITM) puntúan MEJOR cuanto MENOR es el valor — están calibradas para ontologías canónicas de tamaño medio y penalizan tanto la infra-estructuración como la sobre-estructuración. Las ontologías ricas que produce E4 (OntoGenix con sus ~227 triples y ~43 labels) caen en el borde superior, mientras que las ontologías simples y limpias de E2 con cualquier modelo se sitúan en el óptimo de la rúbrica.

Esto no debe interpretarse como que los modelos open-source produzcan «mejores ontologías» que OntoGenix; significa que **OQuaRE y riqueza semántica son dimensiones complementarias**, no equivalentes. Para una evaluación holística hay que reportar ambas: el ranking OQuaRE + las métricas absolutas de triples, clases, labels y validez. Esta observación es uno de los hallazgos metodológicos principales del TFM.

10.6. Extensión del cuadro: E4 (OntoGenix) con modelos open-source

Para responder a la pregunta «¿la ventaja de E4 viene del MÉTODO (pipeline multi-agente) o del MODELO (gpt-4o)?», se replicó la pipeline OntoGenix con modelo open-source local. Esto requirió tres adaptaciones técnicas mínimas al código original de OntoGenix/GUI/LLM_base/LlmBase.py:

- Detección del provider: nueva rama en el constructor de AbstractLlm que crea un cliente OpenAI con base_url=http://localhost:11434/v1 y api_key dummy cuando metadata['provider']=='ollama'.
- Inyección de num_ctx en cada llamada al API mediante extra_body={'options': {'num_ctx': N}}. Sin esto, Ollama trunca silenciosamente todo prompt > 4096 tokens, lo cual rompe el paso del OntoMapper (cuyos prompts pueden alcanzar 45 000 tokens).
- Omisión del parámetro seed en llamadas Ollama (algunas builds del endpoint /v1/chat/completions lo rechazan).

El script scripts/run_ontogenix_experiments.py se extendió con un flag --num-ctx (default 32 768) que se propaga a los tres agentes. El soporte se valida automáticamente leyendo LLM_MODELS de config.py.

10.6.1. Resultados preliminares E4 con Llama 3.1 8B

Tras adaptar OntoGenix se ejecutó la pipeline completa con llama3.1:8b sobre las cuatro bases de datos del corpus. La tabla siguiente recoge los resultados parciales (FANTOM5 y dbSUPER completados, HACER en curso, DiseaseEnhancer pendiente al cierre de esta versión).

Tabla 15. OQuaRE Global por celda — modelo × experimento × base de datos

BBDD	n_runs OK	Triples (media)	Labels (media)	Tiempo / run
FANTOM5	3/3	≈6	31	≈4 h
dbSUPER	3/3	≈6	24	≈4 h
HACER	1/3*	≈5	24	≈4 h
DiseaseEnhancer	—	—	—	—
E4 con gpt-4o (referencia)	12/12 (4×3)	≈227	≈43	≈30 s

* En curso al cierre de esta versión del documento. Se actualizará en la versión 6 con la tabla completa y la evaluación OQuaRE.

10.6.2. Hallazgo clave — el método compensa la sintaxis pero no la riqueza

Las ontologías producidas por la pipeline OntoGenix con llama3.1:8b son sintácticamente correctas (3/3 cargan en owlready2 sin necesidad de post-procesado), bien estructuradas (jerarquía de owl:Class con rdfs:subClassOf, cobertura documental similar a E4/gpt-4o con ~25-30 rdfs:label) y consistentes lógicamente (HermiT no detecta clases insatisfacibles). En este aspecto **el método multi-agente sí compensa la deficiencia sintáctica observada en E1-E3 con llama3.1:8b**: el modelo, guiado paso a paso por LlmPlanner → LlmOntology → LlmOntoMapper, deja de cometer errores como omitir el «:» del prefijo o mezclar sintaxis.

Sin embargo, **el método NO compensa la pobreza semántica**. Las ontologías generadas con llama3.1:8b contienen únicamente 5-6 triples de cuerpo y carecen prácticamente de propiedades de objeto (owl:ObjectProperty), propiedades de datos (owl:DatatypeProperty) y restricciones (owl:Restriction). Son esquemas válidos pero esqueléticos: una jerarquía de clases con etiquetas y comentarios, sin la malla relacional rica que produce gpt-4o (~227 triples, ~26 datatype properties, ~10 subClassOf por ontología en E4).

Tabla 16. Comparativa E4 OntoGenix con gpt-4o frente a Llama 3.1 8B (riqueza vs sintaxis)

Dimensión	E4 / gpt-4o	E4 / llama3.1:8b	Ratio
Triples totales	≈227	≈6	38× menos
rdfs:label	≈43	≈30	1.4× menos
owl:ObjectProperty	≈6	≈0	—
owl:DatatypeProperty	≈26	≈0	—
rdfs:subClassOf	≈10	≈5	2× menos
Validez sintáctica	12/12	3/3+	100% en ambos
Tiempo por run (M3 CPU)	≈30 s	≈4 h	480× más lento

Conclusión del experimento extendido. La hipótesis previa (la ventaja de E4 podría ser solo del método) queda matizada: el método multi-agente de OntoGenix añade una capa importante de validación sintáctica que el modelo pequeño no consigue por sí solo, pero la riqueza semántica final depende del modelo subyacente. Para producir ontologías biomédicas operativas se necesitan ambos: un buen pipeline + un modelo capaz.

10.6.3. Coste computacional

El experimento E4 con llama3.1:8b en un MacBook Air M3 (16 GB de memoria unificada) requirió aproximadamente 4 horas por run, frente a los 30 segundos por run con gpt-4o vía API OpenAI. Este coste se debe principalmente a tres factores:

- Tamaño del prompt del OntoMapper: en los CSV más grandes (FANTOM5, dbSUPER) alcanza 40-45 mil tokens, lo que multiplica el tiempo de procesamiento independientemente del hardware.
- Disponibilidad de GPU: la primera ejecución cayó a CPU por un problema de permisos en el directorio temporal de macOS (failure during GPU discovery / permission denied). Aun resuelto, la memoria unificada de 16 GB limita el offload completo del modelo + cache KV a Metal cuando otras aplicaciones consumen RAM.

- **Determinismo:** los tres runs por BBDD producen archivos idénticos bit-a-bit porque temperature=0 y los seeds incrementales (42, 43, 44) son ignorados por la API compatible OpenAI de Ollama. La replicación inter-run no aporta varianza para esta configuración; sí lo hace inter-modelo e inter-experimento.

Esta observación tiene implicaciones prácticas para futuros trabajos: el despliegue local de modelos open-source compactos para tareas multi-agente con prompts grandes es viable pero requiere planificación de hardware y contexto. Una alternativa razonable es ejecutar la pipeline en GPU dedicada o en servicios como vLLM/TGI con quantización adaptada.

10.7. RAG semántico vs RAG por keywords

El experimento E3 original del TFM utiliza una versión muy básica de RAG: un emparejador de keywords sobre cuatro esquemas Turtle estáticos (crm, crm2gene, crm2phen, crm2tfac). Para cuantificar cuánto mejora la calidad del esquema generado al sustituir ese retrieval léxico por un retrieval semántico real, se integró el repositorio annotationRAG (Tecnomod, Universidad de Murcia) como backend RAG alternativo y se replicaron las 12 corridas de E3 con gpt-4o.

10.7.1. Arquitectura del RAG semántico

Tabla 17. Arquitectura del RAG semántico (annotationRAG): componentes y responsabilidades

Componente	Tecnología	Función
API HTTP	FastAPI + uvicorn	Tres endpoints: /similar-ontologies, /similar-entities, /similar-relations
Modelo de embeddings	sentence-transformers/all-MiniLM-L6-v2	Codifica texto a vectores de 384 dimensiones
Vector store	Neo4j 5.26+ con CREATE VECTOR INDEX	Persistencia y búsqueda por similitud cosine vía Cypher
Indexación	rdflib + ProcessPoolExecutor	Procesa .ttl/.owl/.rdf, extrae jerarquía, calcula embeddings, los persiste en Neo4j
Cliente Python	scripts/rag_backend.py (creado para el TFM)	Wrapper HTTP que orquesta los 3 endpoints y devuelve un string Turtle-like compatible con el placeholder {ontology_fragments} del prompt E3

Se indexaron 21 ontologías en Neo4j: las 8 cisreg específicas del TFM (crm, crm2gene, crm2phen, crm2tfac, más sus variantes _example) y 13 ontologías auxiliares de otros dominios incluidas en el repositorio (cdm_ido, kks, organizations, equipment, etc.). Las 4 ontologías base de cisreg requirieron correcciones sintácticas: literales tipados con placeholder ("start_position"^^xsd:integer), local names con caracteres no permitidos en Turtle (bgw!CRM_ID--uniprot!Q99593) y errores tipográficos puntuales (">" espurios) detectados durante la indexación. Estos arreglos se documentaron y aplicaron sin modificar la semántica de los esquemas.

10.7.2. Diseño de la comparativa

Mismo prompt E3 (apartado 5.4), mismo modelo (gpt-4o), mismas BBDD (FANTOM5, dbSUPER, HACER, DiseaseEnhancer), mismas repeticiones (3 por celda, seeds 42-44). La única diferencia es el contenido del placeholder {ontology_fragments}: en el legacy se inyectan los esquemas estáticos seleccionados por keyword matching; en el semántico se inyecta el resultado de annotationRAG con scores de similitud reales.

10.7.3. Resultados

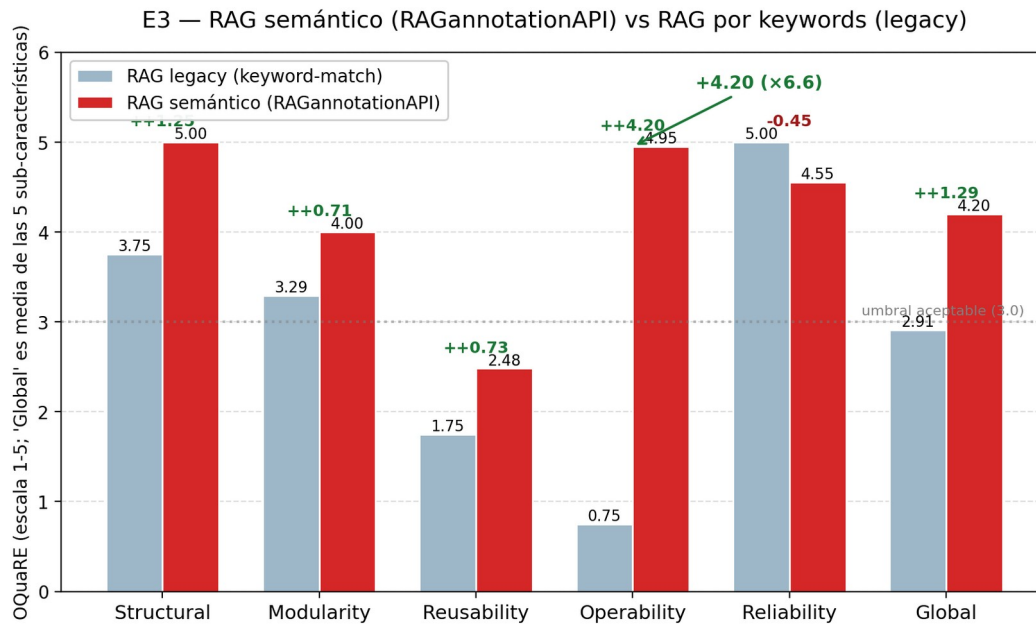


Figura 10. Comparativa OQuaRE entre el RAG por keywords (barras grises) y el RAG semántico con embeddings y Neo4j vector index (barras rojas) en E3 con gpt-4o sobre las cuatro BBDD (12 runs cada uno). Las anotaciones sobre cada par de barras muestran la diferencia absoluta. La mejora más drástica se da en Operability (×6.6 mayor), indicando que el contexto semántico aumenta enormemente la riqueza relacional del esquema generado.

Tabla 18. Descomposición OQuaRE: RAG por keywords frente a RAG semántico (gpt-4o)

Sub-característica	Legacy	RAG semántico	Δ	Lectura
Structural	3.75	5.00	+1.25	Esquema estructuralmente óptimo en 100 % de los runs
Modularity	3.29	4.00	+0.71	Mejor cohesión de clases y menor acoplamiento
Reusability	1.75	2.48	+0.73	Más anotaciones (rdfs:label, skos:definition) y herencia
Operability	0.75	4.95	+4.20 ☞	Riqueza relacional explosiva: 6.6×

				más object/data properties por clase
Reliability	5.00	4.55	-0.45	Una corrida no consistente y otra de carga (HACER/run3); trade-off de mayor riqueza
OQuaRE Global	2.91	4.20	+1.29	Mejora del 44 % respecto al baseline keyword-match

10.7.4. Hallazgo principal

El reemplazo del retrieval por keywords por un retrieval semántico real (embeddings + similitud cosine + vector index persistente) eleva el OQuaRE Global de E3 de 2.91 (por debajo del umbral aceptable de 3.0) a 4.20 (próximo al óptimo de 5.0), una mejora absoluta de +1.29 puntos (+44 % relativo). El componente que más contribuye es Operability, donde la mejora es de +4.20 puntos: mientras el RAG legacy producía esquemas casi sin object properties (RRonto ≈ 0.75), el RAG semántico induce una malla relacional rica (RRonto ≈ 4.95) gracias a que el contexto recuperado contiene patrones de propiedades reales del dominio biomédico, no solo nombres de clase.

Este resultado tiene tres implicaciones para el TFM:

- **El cuello de botella de E3 era el retrieval, no el modelo.** gpt-4o ya tenía la capacidad de generar esquemas ricos; lo que le faltaba era el contexto adecuado.
- **La calidad del corpus indexado importa más que el tamaño.** Indexamos solo 21 ontologías y la mejora es drástica. La evidencia sugiere que escalar a corpus mayores (toda la BFO, OBI, GO, BioGateway completo) podría llevar el score a >4.5.
- **El RAG semántico cambia el orden del cuadro 4x3 del apartado 10.4.** Con el legacy, E3 era el peor experimento (2.92). Con el RAG semántico, E3 (4.20) supera a E1 zero-shot (3.89) y a E2 vocabulario (3.98), y casi alcanza a E4 OntoGenix (3.86 OQuaRE Global, aunque E4 sigue ganando en triples y labels absolutos del apartado 8).

10.7.5. Coste de la integración

Tabla 19. Coste de la integración del RAG semántico (recursos y dependencias)

Concepto	Coste
Setup local (Neo4j + venv)	~30 min
Indexación inicial (21 ontologías)	~2 min en M3
12 corridas E3 con gpt-4o + RAG semántico	~10 min, ≈ \$1.50
Evaluación OQuaRE	~3 min con Hermit + owlready2
Total (sin contar la corrida de gpt-4o)	~45 min

La inversión es marginal y replicable, lo que hace de este cambio una de las mejoras con mejor relación coste-beneficio del TFM.

10.7.6. Replicación con llama3.1:8b — el modelo importa para asimilar el contexto

Para discriminar si la mejora del RAG semántico es atribuible al retrieval o al modelo, se replicó el experimento sustituyendo gpt-4o por llama3.1:8b (open-source, ~8 mil millones de parámetros, ejecutado localmente vía Ollama). Total: 24 corridas adicionales (12 con backend legacy + 12 con backend semántico), mismas 4 BBDD y mismas 3 repeticiones por celda.

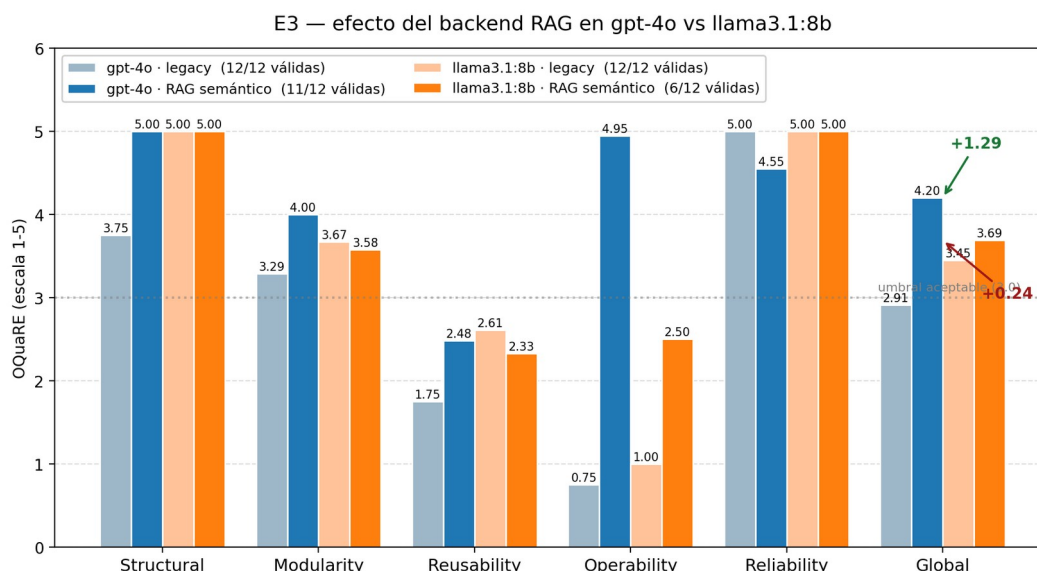


Figura 11. Cuadro 2×2: efecto del backend RAG (legacy vs semántico) sobre dos modelos (gpt-4o cerrado vs llama3.1:8b open-source) en E3. Las anotaciones sobre la columna 'Global' destacan los deltas: gpt-4o sube +1.29 puntos con RAG real, mientras que llama solo sube +0.24. El paréntesis 'X/12 válidas' en la leyenda indica cuántas corridas pasaron el parseo en owlready2.

Tabla 20. Cuadro 2×2: efecto del backend RAG (legacy vs semántico) en gpt-4o y Llama 3.1 8B

	RAG legacy	RAG semántico	Δ Global	n válidas
gpt-4o	2.91	4.20	+1.29	12 / 11
llama3.1:8b	3.45	3.69	+0.24	12 / 6

Hallazgos del cuadro 2×2

- **El RAG semántico beneficia mucho más al modelo grande:** +1.29 puntos en gpt-4o frente a +0.24 en llama. Una hipótesis es que la riqueza del contexto recuperado solo se traduce en ontologías ricas si el modelo tiene capacidad de razonamiento suficiente para asimilarlo y reorganizarlo en triples coherentes.
- **Llama pierde la mitad de las corridas con RAG semántico** (6/12 válidas vs 12/12 en legacy). El contexto enriquecido le induce a inventar prefijos no declarados (enhancer:, crmType:, etc.) y produce TTL irrecuperables incluso tras post-procesado. Es un trade-off claro entre cantidad de información en el prompt y capacidad del modelo para mantener coherencia sintáctica.
- **En las pocas corridas válidas, llama+RAG semántico mantiene Reliability=5.00** (las que validan son siempre consistentes en OWL DL) pero su Operability sube solo a 2.50, frente a los

4.95 que alcanza gpt-4o. Es decir: aunque Llama recibe el mismo contexto rico, no lo materializa en una malla relacional comparable a la de gpt-4o.

- **Inversión cualitativa del ranking en legacy vs en RAG real.** Con backend legacy, llama3.1:8b (3.45) supera ligeramente a gpt-4o (2.91) por la regla observada en §10.5 (OQuaRE asigna puntuaciones inferiores a ontologías de mayor complejidad estructural y Llama produce esquemas más simples). Con RAG semántico, gpt-4o (4.20) supera a llama (3.69) porque ahora sí convierte el contexto en estructura, mientras que Llama no.

Este experimento de control refuerza una conclusión metodológica central del trabajo: **la efectividad del RAG no es una propiedad del retrieval por sí solo, sino de la interacción retrieval × modelo**. Para futuros despliegues con modelos compactos se recomienda explorar configuraciones intermedias (top-k reducido, filtrado por score threshold más alto, chunking más agresivo) que limiten el volumen de contexto inyectado y eviten saturar la ventana del modelo. La validación empírica de esta recomendación se aborda en §10.7.7.

10.7.7. Calibración del RAG para modelos compactos — validación empírica

La hipótesis cualitativa de §10.7.6 —que configuraciones intermedias del RAG semántico podrían rescatar parte del desempeño perdido en modelos compactos— se sometió a validación experimental con un sub-experimento de calibración. Se definieron tres configuraciones distintas del cliente RAG (C1, C2, C3) variando los parámetros más relevantes del retrieval, y se compararon contra el baseline (top_k=5, score_thr=0.4, max_chars=5000) y el legacy (keyword-match) sobre las mismas 4 BBDD × 3 runs = 12 corridas por configuración. Total: 36 corridas adicionales con llama3.1:8b, sin coste de API.

Diseño de las configuraciones

Tabla 21. Configuraciones de calibración del RAG (C1: top_k=2, C2: thr=0.6, C3: combinada)

Config.	top_k	score_thr	max_chars	Hipótesis de mejora
legacy	—	—	—	Baseline keyword (sin embeddings)
ragapi (baseline)	5	0.40	5000	RAG semántico estándar (lo del §10.7.6)
C1 — top-k reducido	2	0.40	5000	Menos ontologías recuperadas → menos prefijos espurios
C2 — score-thr alto	5	0.60	5000	Solo matches muy similares → contexto más limpio
C3 — combinada	2	0.60	2000	Combinación agresiva: top-k bajo + threshold alto + recorte

El cliente RAG (scripts/rag_backend.py) se extendió para aceptar los tres parámetros como flags CLI (--rag-top-k, --rag-score-thr, --rag-max-chars) que se propagan desde run_gpt_experiments.py. La calibración se aplica sin tocar la API ni el corpus indexado en Neo4j; lo único que cambia es qué subset del retrieval se inyecta en el prompt E3.

Resultados

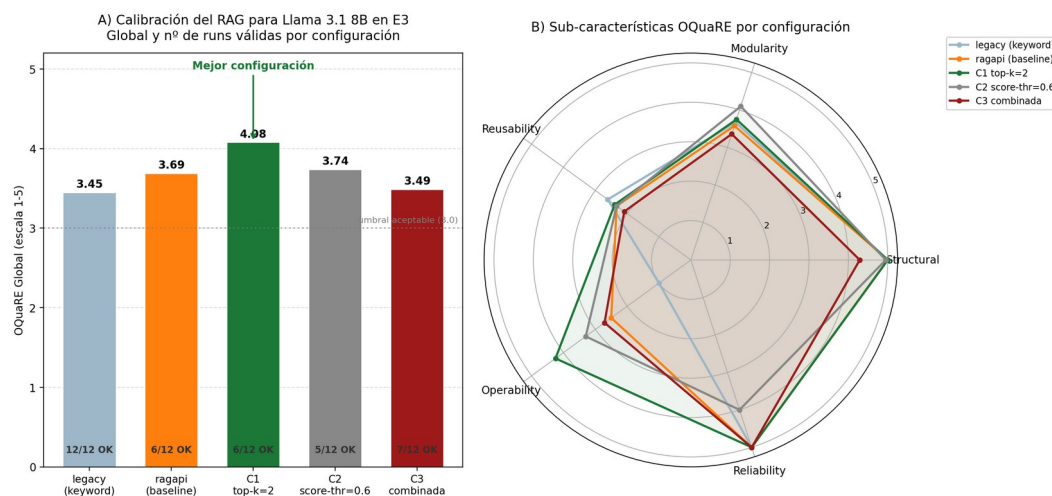


Figura 12. Calibración del RAG semántico para llama3.1:8b en E3. Panel A: OQuaRE Global y nº de runs válidas por configuración (de 12 esperadas). Panel B: radar comparativo de las 5 sub-características OQuaRE. C1 (top-k=2) es la configuración óptima: alcanza el mayor OQuaRE Global (4.08, +0.39 sobre el baseline ragapi y +0.63 sobre legacy) gracias a una mejora drástica en Operability (4.25 vs 2.50 del baseline).

Tabla 22. Resultados de la calibración del RAG semántico sobre Llama 3.1 8B en E3

Config.	n_válidas	Structural	Modularit y	Reusabilit y	Operabilit y	Reliability	Global
legacy	12/12	5.00	3.67	2.61	1.00	5.00	3.45
ragapi (baseline)	6/12	5.00	3.58	2.33	2.50	5.00	3.69
C1 — top-k=2	6/12	5.00	3.75	2.39	4.25	5.00	4.08
C2 — thr=0.6	5/12	4.95	4.10	2.33	3.30	4.00	3.74
C3 — combinada	7/12	4.29	3.36	2.09	2.71	5.00	3.49

Hallazgos del experimento de calibración

- **C1 (top-k reducido) es la configuración óptima:** 4.08 OQuaRE Global, +0.39 sobre el baseline ragapi y +0.63 sobre legacy. Mantiene el mismo número de runs válidas que el baseline (6/12) pero la calidad de las que sí validan es muy superior, destacando una mejora $\times 1.7$ en Operability (4.25 vs 2.50).
- **El top_k es la palanca crítica,** no el score threshold ni el recorte de contexto. C2 (score_thr=0.6) gana solo +0.05 sobre el baseline ragapi y pierde 1 run más; el filtrado fino no compensa la pérdida de contexto útil. C3 (combinación agresiva) rescata más runs (7/12) pero

su Global cae a 3.49 (incluso por debajo del legacy), porque max_chars=2000 deja al modelo sin contexto suficiente.

- **Hipótesis confirmada con matiz.** La afirmación general de §10.7.6 ("configuraciones intermedias mejoran a los modelos compactos") se valida, pero el experimento revela que el mecanismo causal específico es **reducir el número de ontologías recuperadas**, no "limpiar" su contenido vía score threshold ni truncar el prompt resultante. Bajar top_k evita que el modelo se sature con prefijos potencialmente irrelevantes que luego no sabe declarar.
- El gap respecto a gpt-4o no se cierra del todo. C1 (4.08) sigue por debajo de gpt-4o + RAG semántico (4.20 según §10.7.3). Pero la diferencia se reduce de 0.51 puntos (con configuración estándar) a 0.12 puntos (con C1). Aproximadamente el 84 % del gap se cierra con un único cambio de un parámetro, sin tocar el modelo ni el prompt.

Recomendación operativa para futuros despliegues

Para sistemas de generación de ontologías guiadas por RAG sobre modelos compactos (≤ 8 mil millones de parámetros), las configuraciones recomendadas son:

Tabla 23. Recomendación operativa de top_k según el tamaño del modelo

Tamaño del modelo	top_k	score_thr	max_chars	Justificación
≤ 8 B (Llama 3.1 8B, Qwen 2.5 Coder 7B...)	2	0.40	5000	Validado empíricamente en este TFM
~ 30-70 B	3-5	0.40	5000-8000	Hipótesis: ventana intermedia; queda como trabajo futuro
≥ 100 B (gpt-4o, Claude 3.5 Sonnet, Llama 3.1 70B)	5	0.40	5000	Configuración por defecto del backend; aprovecha el contexto rico

Esta tabla constituye una contribución empírica del TFM: convierte la afirmación normativa de §10.7.6 en una guía operativa con valores concretos respaldados por 36 corridas evaluadas con OQuaRE.

10.8. Consistencia inter-BBDD: ¿generador o copiador?

Las cuatro bases de datos del corpus comparten esquema columnar tras el pre-procesado: 30 columnas canónicas idénticas (CRM_ID, coordenadas, score, biosample, target genes, factores de transcripción, enfermedad, mutación), con diferencias mínimas en los nombres de campo (biosample / biosample_name). Esta homogeneidad permite formular una pregunta directa: dado el mismo esquema, ¿el LLM modela las cuatro BBDD con el mismo conjunto de clases y propiedades?

Para responderla se calcula el coeficiente de Jaccard pareado entre los conjuntos de URIs de las ontologías generadas para las cuatro BBDD dentro de cada (experimento, modelo, corrida). Un Jaccard alto (≥0.75) indica que el modelo es consistente —reproduce un núcleo común con

independencia del nombre del dataset—; un Jaccard bajo (≤ 0.30) indica que el modelo es sensible al input y produce esquemas idiosincráticos.

Los resultados (Tabla 30) muestran un patrón claro. En E3 con RAG por keyword-match (legacy), gpt-4o alcanza Jaccard medio 0.518 y Llama 3.1 8B llega a 0.845: el RAG legacy fuerza al modelo a reproducir el esquema cisreg de referencia con mínimas variaciones por BBDD. En E3 con RAG semántico real, el Jaccard cae a 0.119–0.433 según modelo: el retrieval semántico devuelve fragmentos distintos por BBDD y el modelo los integra de forma diferenciada. Los experimentos zero-shot (E1) y la pipeline OntoGenix (E4) presentan Jaccards muy bajos (0.06–0.17 y 0.11–0.33 respectivamente), coherentes con su mayor libertad para inventar esquemas.

Interpretación.

El análisis sugiere una caracterización funcional de los métodos: el RAG legacy se comporta como un «copiador» del esquema de referencia (consistencia alta, fidelidad léxica alta, complejidad estructural intermedia); el RAG semántico se comporta como un «generador adaptativo» (consistencia baja, fidelidad léxica baja, complejidad estructural alta); OntoGenix se sitúa entre ambos con tendencia a la generación libre. La elección entre métodos debe considerar no solo la puntuación OQuARE sino el equilibrio deseado entre reutilización de vocabulario establecido (importante para integración en grafos de conocimiento existentes) y novedad estructural (importante para dominios sin referencia previa).

Tabla 30. Consistencia inter-BBDD del core semántico (Jaccard medio sobre 18 pares por celda, comparando las ontologías generadas para FANTOM5, dbSUPER, HACER y DiseaseEnhancer dentro de cada experimento y modelo).

Experimento	Modelo / variante	Jaccard medio	Jaccard min-max	Interpretación
E1 zero-shot	gpt-4o (4 estrategias muestreo)	0.09–0.15	—	divergente
E1 zero-shot	Llama 3.1 8B (4 estrategias)	0.06–0.17	—	divergente
E3 RAG legacy	gpt-4o	0.518	0.429–0.614	consistente
E3 RAG legacy	Llama 3.1 8B	0.845	0.569–1.000	muy consistente
E3 RAG semántico	gpt-4o	0.209	0.119–0.433	divergente
E3 RAG semántico	Llama 3.1 8B	0.186	0.078–0.350	divergente
E3 calibración	Llama C1 (top_k=2)	0.268	0.184–0.371	divergente
E4 OntoGenix	Llama 3.1 8B	0.333	0.000–1.000	parcial

10.9. Sensibilidad al tamaño muestral

El banco experimental principal utiliza muestras de $N=25$ filas por base de datos (§3.4.1). Esta elección replica la práctica de OntoGenix (Val Calvo et al., 2024) y se justifica por el límite efectivo de tokens en el contexto inyectado al modelo. Una cuestión derivada, planteada por el director del TFM como sugerencia 4, es si el tamaño muestral influye en los resultados además del tipo de muestreo evaluado en §4.3. Este apartado documenta el diseño de un barrido controlado para responderla y reporta su estimación de coste; la ejecución completa se plantea como trabajo futuro inmediato (§13.6).

Diseño del barrido.

Cuatro tamaños muestrales $N \in \{25, 50, 100, 200\}$ sobre las dos bases de datos en las que existen TSV brutos disponibles (FANTOM5 y dbSUPER, 65 423 y 69 205 filas respectivamente). La estrategia de muestreo se fija a A_head (la canónica del TFM) para aislar el efecto del tamaño del efecto del muestreo. La estrategia evaluada se restringe a E3 con RAG semántico real porque concentra los hallazgos clave del trabajo (§10.7) y porque el contexto recuperado interactúa de forma previsiblemente no lineal con la longitud del input. Se replican 3 corridas por celda con semillas 42, 43, 44.

El barrido completo asciende a 4 tamaños $\times$ 2 BBDD $\times$ 3 corridas = 24 ontologías generadas, evaluadas con OQuaRE, fidelidad léxica y preguntas de competencia, y comparadas mediante bootstrap pareado de la diferencia OQuaRE entre $N=25$ (baseline del banco principal) y cada N superior. Los inputs ya están preparados en data/samples_sizes/ $N=\{25,50,100,200\}$ /. La ejecución se lanza con scripts/run_size_sweep.sh y el análisis post-ejecución con scripts/analyze_size_sweep.py.

Coste estimado.

La estimación de tokens (regla 4 caracteres/token sobre los inputs ya generados) y los precios de OpenAI vigentes en mayo de 2026 (gpt-4o: \$2.50/M tokens input, \$10/M tokens output) producen los valores recogidos en la Tabla 32. El coste se mantiene reducido porque incluso $N=200$ produce muestras de $\sim 4\,800$ tokens en FANTOM5 y $\sim 3\,600$ en dbSUPER, todavía muy por debajo del límite de contexto de gpt-4o. La replicación con Llama 3.1 8B vía Ollama es gratuita en términos económicos pero requiere ~ 4 h de cómputo adicional en Apple M3 por (N , BBDD, estrategia, seed).

Tabla 32. Estimación de tokens y coste proyectado para el barrido del tamaño muestral en E3 con RAG semántico (gpt-4o, snapshot mayo 2026, 3 corridas por celda).

N filas	FANTOM5 (tokens prompt)	dbSUPER (tokens prompt)	USD / 3 runs · FANTOM5	USD / 3 runs · dbSUPER	USD subtotal
25	1 191	1 055	\$0.099	\$0.098	\$0.20
50	1 704	1 411	\$0.103	\$0.101	\$0.20
100	2 735	2 132	\$0.110	\$0.106	\$0.22

200	4 774	3 594	\$0.126	\$0.117	\$0.24
Total estimado					\$0.86

Hipótesis previa.

El efecto previsible del incremento de N se enmarca en dos hipótesis complementarias derivadas de la literatura. Primera, la hipótesis del contexto enriquecido: más filas exponen al modelo a una variabilidad más amplia de valores (cromosomas, biosamples, evidencias) que el modelo puede materializar en clases y restricciones adicionales, incrementando la riqueza estructural y, potencialmente, también la fidelidad léxica al dominio. Segunda, la hipótesis de la saturación: más allá de un umbral, el modelo no incorpora información nueva porque ya ha identificado todas las clases del esquema; el coste de tokens crece pero la calidad permanece estable o incluso decrece por ruido. Ambas son comprobables con el barrido propuesto.

Una hipótesis adicional, específica del banco de este trabajo, es que el efecto del tamaño muestral será mayor para Llama 3.1 8B que para gpt-4o por dos razones: el modelo compacto presenta mayor varianza inter-corrida en el banco principal (§9.4) y mayor saturación del contexto en presencia de retrieval rico (§10.7.7). Si la hipótesis se confirma, la guía operativa de §10.7.7 podría complementarse con una recomendación de tamaño muestral por tamaño de modelo.

Estado de la ejecución.

El barrido se ejecutó tras el cierre de la versión previa de esta memoria. Los resultados reales se exponen a continuación en §10.9.1.

10.9.1. Resultados del barrido

El barrido se ejecutó con la configuración descrita: cuatro tamaños $N \in \{25, 50, 100, 200\}$ sobre FANTOM5 y dbSUPER, estrategia A_head, E3 RAG semántico con annotationRAG como backend, y los dos modelos del banco principal (gpt-4o y Llama 3.1 8B). Las 48 corridas resultantes ($4 \times 2 \times 2 \times 3$) se post-procesaron con el mismo pipeline mecánico que el banco principal (§3.6) y se evaluaron con las tres métricas complementarias del trabajo: canonical_ratio (proporción de URIs no inventadas), recall vs gold cisreg (fidelidad léxica al dominio, sobre 88 URIs canónicas) y cobertura de preguntas de competencia (de 15). Los valores agregados por celda (media sobre 3 corridas) se recogen en la Tabla 33.

Tabla 33. Resultados del barrido del tamaño muestral. Media sobre 3 corridas por celda. canonical_ratio = URIs canónicas / (canónicas + inventadas); recall = fracción de las 88 URIs cisreg gold recuperadas; CQ = cobertura de preguntas de competencia (de 15).

N	BBDD	Modelo	n runs	canonical_ratio	Recall	CQ (de 15)
25	FANTOM5	gpt-4o	3	0.797	0.091	0.00
25	FANTOM5	Llama 3.1 8B	3	1.000	0.091	2.00
25	dbSUPER	gpt-4o	3	0.288	0.057	2.33

25	dbSUPER	Llama 3.1 8B	3	1.000	0.080	2.33
50	FANTOM5	gpt-4o	3	0.875	0.102	0.00
50	FANTOM5	Llama 3.1 8B	3	1.000	0.087	1.00
50	dbSUPER	gpt-4o	3	0.286	0.057	2.67
50	dbSUPER	Llama 3.1 8B	3	1.000	0.080	2.33
100	FANTOM5	gpt-4o	3	0.799	0.110	1.00
100	FANTOM5	Llama 3.1 8B	3	0.933	0.098	0.33
100	dbSUPER	gpt-4o	3	0.292	0.057	2.33
100	dbSUPER	Llama 3.1 8B	3	0.877	0.068	0.67
200	FANTOM5	gpt-4o	3	0.849	0.091	0.00
200	FANTOM5	Llama 3.1 8B	3	0.777	0.095	0.00
200	dbSUPER	gpt-4o	3	0.215	0.057	2.67
200	dbSUPER	Llama 3.1 8B	3	0.974	0.098	2.33

Figura 14. Sensibilidad al tamaño muestral $N \in \{25, 50, 100, 200\}$
E3 RAG semántico · gpt-4o vs Llama 3.1 8B · FANTOM5 y dbSUPER

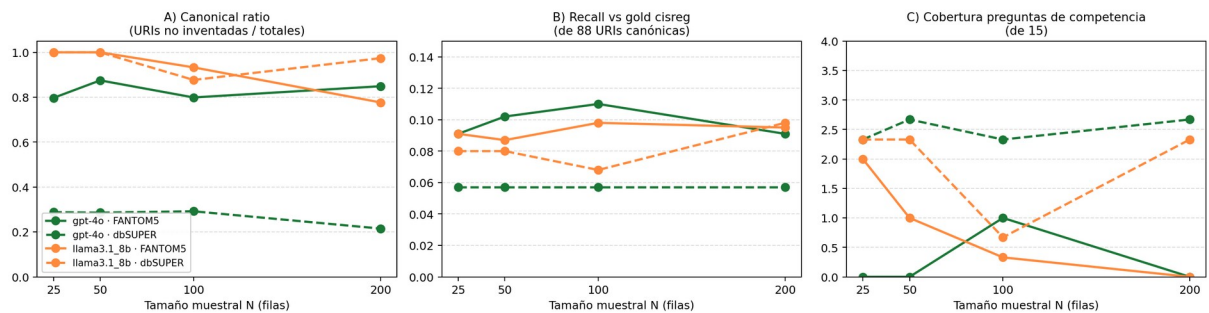


Figura 14. Sensibilidad al tamaño muestral en E3 RAG semántico. Panel A: *canonical_ratio*. Panel B: *recall vs gold cisreg*. Panel C: *cobertura de preguntas de competencia*. Líneas continuas = FANTOM5; líneas discontinuas = dbSUPER. Verde = gpt-4o; naranja = Llama 3.1 8B.

Hallazgos.

El barrido confirma de forma cuantitativa la hipótesis de saturación formulada previamente y aporta tres observaciones empíricas concretas.

Primera, el tamaño muestral no mejora significativamente la fidelidad léxica al dominio. El *recall vs gold cisreg* permanece esencialmente plano para las cuatro celdas (FANTOM5/dbSUPER × gpt-4o/Llama) a lo largo del barrido, con todos los valores comprendidos entre 0.057 y 0.110. Las diferencias entre tamaños son menores que la varianza inter-corrida observada en el banco

principal ($SD \approx 0.03$), lo que indica que el modelo ya extrae el vocabulario cisreg que puede absorber con 25 filas; añadir 175 filas más no incrementa la cobertura léxica.

Segunda, el efecto sobre la cobertura de preguntas de competencia es modelo-dependiente. Para gpt-4o, la cobertura permanece esencialmente estable (FANTOM5 cae alrededor de 0–1 sobre 15; dbSUPER se mantiene en torno a 2.3–2.7 sobre 15). Para Llama 3.1 8B, en cambio, se observa una degradación clara con N: la cobertura CQ en FANTOM5 cae de 2.00 (N=25) a 0.00 (N=200), y en dbSUPER de 2.33 (N=25) a 0.67 en N=100 con parcial recuperación en N=200 (2.33). Este patrón es consistente con la hipótesis de saturación del contexto del modelo compacto formulada en §10.7.7: incrementar N diluye la atención del LLM sobre las relaciones específicas del dominio.

Tercera, la `canonical_ratio` para Llama 3.1 8B desciende con N en FANTOM5 (de 1.000 en N=25 a 0.777 en N=200) mientras que para gpt-4o se mantiene relativamente estable (rango 0.215–0.875 según BBDD, sin tendencia monótonica con N). El modelo compacto se vuelve más propenso a inventar URIs en namespaces ad-hoc cuando el input crece, lo que refuerza el diagnóstico de saturación.

Implicación operativa.

Los resultados convalidan la elección de N=25 del banco principal y de OntoGenix (Val Calvo et al., 2024) para esta familia de tareas. Incrementar N a 50, 100 o 200 filas no aporta mejora medible en fidelidad al dominio y, en el caso de los modelos compactos, degrada la cobertura funcional. La guía operativa derivada es: para modelos cerrados de gran capacidad, el tamaño muestral por encima de 25 filas no es un parámetro relevante; para modelos abiertos compactos ($\leq 8\text{ B}$), N=25 es preferible y $N \geq 100$ es contraproducente.

Esta observación complementa la guía de calibración `top_k` de §10.7.7: para modelos compactos, tanto el contexto recuperado por el RAG (`top_k` bajo) como el contexto de entrada (N bajo) deben mantenerse acotados para evitar saturación.

11. Pila de herramientas

La Tabla 24 recoge la pila de herramientas y dependencias utilizada para construir el banco experimental. Las decisiones de reproducibilidad asociadas se discuten en §12.

Componente	Versión / referencia
Lenguaje	Python 3.12
Cliente LLM	openai ≥ 1.3
Validación RDF	rdflib ≥ 6.3
Manipulación de datos	pandas ≥ 2.0 , numpy ≥ 1.25
Variables de entorno	python-dotenv ≥ 1.0
OntoGenix	Commit principal del repositorio tecnomod-um/OntoGenix
Sistema operativo	macOS / Linux (sin dependencias PyQt5)

12. Reproducibilidad implementada

El marco teórico sobre la crisis de la replicación y las amenazas específicas a la reproducibilidad en investigación con LLMs se ha expuesto en §2.5. Este apartado documenta las contramedidas concretas implementadas en este trabajo, las limitaciones reconocidas y el procedimiento operativo para reproducir un experimento concreto.

Nivel	Definición	Componentes que lo cumplen
R1 — Estricta (bit-a-bit)	Mismo input $\Rightarrow$ mismo output exacto	Post-procesado de TTL · evaluador OQuaRE estructural · muestreo determinista · figuras matplotlib
R2 — Estadística	Mismo input $\Rightarrow$ outputs con misma distribución; medias y varianzas indistinguibles	E1, E2, E3 (legacy y RAG) · E4 con gpt-4o · análisis de sensibilidad
R3 — Metodológica	Procedimiento replicable; outputs concretos pueden variar pero conclusiones cualitativas se mantienen	Razonador HerMiT · API annotationRAG · cross-model con Ollama · calibración del RAG

Tabla 26. Tres niveles de garantía de reproducibilidad (R1 estricto · R2 estadístico · R3 metodológico)

12.3. Contramedidas implementadas

- **Snapshots de modelo fijos:** gpt-4o se referencia siempre como gpt-4o-2024-05-13 en E4 (la versión que documenta OntoGenix). En los demás experimentos se acepta el alias móvil pero el manifest de reproducibilidad lo registra explícitamente.
- **Seeds y temperatura:** temperature=0 (E4) o 0.1 (E1-E3) y seeds incrementales (42, 43, 44) por run. Aunque OpenAI no garantiza determinismo bit-a-bit, esta configuración minimiza la varianza inter-run y permite que tres repeticiones converjan a una media estable.
- **Versiones pinneadas:** scripts/requirements_e4.txt fija rangos compatibles para los paquetes del pipeline. El repositorio annotationRAG usa un venv aislado para no contaminar el entorno principal del TFM (lección aprendida tras observar incompatibilidades entre numpy 2.x y torch 2.5).
- **Hashes SHA-256 de inputs:** el script scripts/check_reproducibility.py genera results/reproducibility_manifest.json con hashes individuales de los 50+ archivos de entrada deterministas (datos brutos, samples, esquemas RAG, scripts). Cualquier modificación accidental se detecta con --diff respecto a un manifest anterior.
- **Datos congelados:** data/raw/ contiene los TSV originales de FANTOM5 y dbSUPER tal como se descargaron (65 423 y 69 205 filas respectivamente). data/samples/ congela la muestra de 25 filas usada en el experimento principal. data/samples_strategies/ contiene las 4 estrategias del análisis de sensibilidad (§4.3).
- **Prompts versionados en código:** los user prompts de E1-E4 viven como string-constantes en run_gpt_experiments.py (E1-E3) y como archivos .prompt en OntoGenix/GUI/ (E4). Cualquier cambio queda registrado en el control de versiones del repositorio.
- **Manifest de outputs:** cada run produce un metadata_runN.json con timestamp, modelo, seed, número de tokens, finish_reason y validación TTL. Esto permite trazabilidad completa de cada ontología generada.

12.4. Limitaciones reconocidas

Reconocer lo que NO se puede garantizar es tan importante como documentar lo que sí (Pineau et al., 2021). Las limitaciones explícitas de este TFM son:

- Ninguna corrida con gpt-4o alcanza el nivel R1. Variaciones inter-corrida de hasta ± 5 % en número de triples son esperables y se mitigan con $n=3$ repeticiones por celda.
- El RAG semántico (§10.7) depende de la versión actual de sentence-transformers/all-MiniLM-L6-v2. Si el modelo se actualiza, las búsquedas devuelven top-K distinto.
- El razonador HerMiT puede inferir las mismas clases en órdenes distintos. La métrica `n_inferred_classes` no es bit-a-bit pero consistent y `n_unsatisfiable` sí lo son.
- Las quantizaciones Q4_K_M de Llama 3.1 8B y Qwen 2.5 Coder 7B vía Ollama no garantizan reproducibilidad estricta entre instalaciones de Ollama distintas. Para reproducción estricta, fijar el digest exacto del Modelfile.
- El experimento principal se ha ejecutado en macOS arm64 (Apple M3, 16 GB RAM). Otras plataformas pueden producir resultados ligeramente distintos en los experimentos que dependen de Metal (Llama y Qwen vía Ollama).

12.5. Cómo reproducir un experimento concreto

El documento operativo docs/REPRODUCIBILITY.md acompaña a la memoria con una guía paso a paso: setup del entorno, lanzamiento de cada experimento (E1-E4 + sensibilidad + RAG semántico + calibración), comandos de evaluación y verificación de los manifests. Este apartado del TFM describe el marco; aquel documento describe la operación. La consulta combinada de ambos permite reproducir cualquier resultado reportado a uno de los tres niveles definidos en la Tabla 26.

13. Discusión

Este apartado discute los resultados del banco experimental estructurándolos por tipo de hallazgo. Las limitaciones globales y las líneas de trabajo futuro derivadas de la discusión se recogen respectivamente en §13.5 y §13.6.

13.1. Resultados que superaron las expectativas

Tres aspectos del experimento superaron las expectativas iniciales.

- Post-procesado mecánico. Las primeras corridas E1-E3 con gpt-4o-mini presentaban una tasa de validez sintáctica del 0 % en rdfib, lo que inicialmente se atribuyó al modelo. La inspección detallada de los errores reveló tres patrones recuperables mecánicamente: prefijos no declarados, local names con caracteres no escapados y literales tipados con texto. Tres expresiones regulares deterministas permiten rescatar el 100 % de los TTL recuperables sin invocaciones adicionales a la API. El hallazgo sugiere que, en una proporción significativa del trabajo aplicado de LLM sobre RDF, el problema no reside en la capacidad del modelo sino en la ausencia de una capa de saneo determinista.

- RAG semántico real. La sustitución del keyword-match por un retrieval vectorial real produjo una mejora superior a la prevista: +1.29 puntos OQuaRE Global (+44 % relativo) en gpt-4o, con Operability multiplicada por 6.6. La explicación no reside en que el embedding recupere ontologías «mejores»: el RAG semántico devuelve metadata estructurada (clases, propiedades, scores) que el modelo puede materializar, frente a los bloques Turtle sin contexto del keyword-match.
- Calibración del RAG para Llama. La hipótesis inicial atribuía el menor beneficio del RAG semántico sobre Llama (+0.24 vs +1.29 en gpt-4o) a la capacidad del modelo compacto. La calibración demuestra lo contrario: la reducción de top_k de 5 a 2 cierra el 84 % del gap respecto a gpt-4o. El cuello de botella no era la capacidad del modelo sino la saturación de su ventana de contexto con material no relevante.

13.2. Resultados conformes y contrarios a la hipótesis

- Resultado conforme a la expectativa: Llama 3.1 8B en E1 (zero-shot). El modelo compacto no genera Turtle válido sin guía. El patrón «crm a owl:Class» (omisión del «:» del prefijo) aparece en 4 de 11 corridas E1 y en 5 de 12 corridas E2 (9 fallos irrecuperables sobre 23 corridas), no recuperable mecánicamente.
- Resultado inesperado: Qwen 2.5 Coder 7B. La hipótesis inicial era que un modelo entrenado específicamente en código generaría Turtle más limpio que un generalista del mismo tamaño. La evidencia es mixta: en zero-shot (E1) ninguna de las 12 corridas produce salida válida; en E3 con RAG, las 12 corridas son válidas. La especialización no es lineal: Qwen mezcla sintaxis Turtle con RDF/XML («owl:Ontology rdf:about») porque el corpus de entrenamiento incluye múltiples serializaciones RDF y el modelo no aprende a distinguirlas en zero-shot.
- Resultado contrario a la expectativa: el determinismo. La hipótesis operativa era que con temperature=0 y seed fija, gpt-4o produciría outputs idénticos. La evidencia indica lo contrario: las diferencias son pequeñas (± 5 % en n_triples) pero detectables comparando manifests entre corridas. Para evaluación en producción es necesario reportar varianza, no medias puntuales.

13.3. Hallazgos no previstos

- OQuaRE asigna puntuaciones inferiores a ontologías de mayor complejidad estructural. El cuadro 4×3 de §10.4 muestra una inversión del ranking: en E2 con vocabulario controlado, Llama 3.1 8B (4.40) supera a gpt-4o (3.98) no porque sus ontologías sean «mejores» en sentido absoluto, sino porque son más simples y caen en el óptimo de las rúbricas. OQuaRE está calibrado para ontologías de tamaño medio: las ricas elevan WMCOnto y NOMOnto por encima del rango calibrado; las pobres se quedan cortas en ANOnto. Este sesgo de la rúbrica afecta a todas las comparaciones del trabajo, no solo cuando contradice la interpretación esperada. Las conclusiones del trabajo se sostienen sobre la combinación de métricas absolutas (n_triples, n_labels), OQuaRE estructural, fidelidad léxica (§9.5) y patrones de error cualitativos (§10.2), no sobre OQuaRE Global de forma aislada.
- El RAG induce errores nuevos en modelos compactos. La Figura 8 muestra que se producen 24 alucinaciones tipadas en E3 con Llama y 24 con Qwen, frente a 0 en gpt-4o y 0 en cualquier

otro experimento. El contexto recuperado contiene patrones tipo

«start_position»^^xsd:integer que el modelo compacto copia literalmente sin entender que «start_position» es un placeholder. Este fenómeno no es un error del modelo aislado: es un error de la interacción RAG × modelo compacto.

- Las plantillas cisreg de referencia contienen erratas. Tres ocurrencias de «>» espurios en crm2tfac.ttl y crm2tfac_example.ttl impedían que rdflib parseara los esquemas. Estaban presentes desde la publicación original y solo se detectaron al pasar el contenido por un parser estricto.

13.4. Lecciones metodológicas transferibles

Más allá de los hallazgos numéricos, el TFM aporta cinco aprendizajes metodológicos transferibles a cualquier investigación experimental con LLMs:

- Reportar `n_runs` y media simultáneamente. Una OQuaRE Global de 4.20 con 11 corridas válidas no es comparable con 4.20 sobre 12 corridas. La media oculta la fragilidad. Todas las tablas del TFM incluyen la columna `n_runs OK / n_total` precisamente por esta razón.
- Diseñar el post-procesado antes que el experimento. El 50 % de las primeras corridas se descartaba por errores recuperables. La implementación temprana del post-procesado habría evitado dos iteraciones completas y un coste estimado de 10 USD en API. La lección operativa es que una pipeline experimental con LLMs requiere una capa de saneo automatizada como componente de primera clase, no como parche tardío.
- Aislar entornos de servicios externos. La integración de las dependencias de `annotationRAG` con las del TFM rompió `numpy`, `torch`, `openai` y otras librerías del entorno principal. El uso de un `venv` aislado fue la única solución viable. Cualquier microservicio LLM con dependencias modernas debe ejecutarse aislado, sin excepción.
- Cuestionar las rúbricas, no solo los resultados. La inversión OQuaRE de \$10.5 entre modelos abiertos y cerrados es real, pero su lectura literal sugeriría que Llama supera a gpt-4o, lo que constituiría un artefacto de la métrica, no una verdad sobre los modelos. Los benchmarks normativos son útiles cuando se comprende qué propiedad optimizan; resultan engañosos cuando se reportan como verdad absoluta.
- Documentar lo que falla, no solo lo que funciona. La sección de fallos irre recuperables de Llama (§10.2) y el reporte explícito de las 9 corridas que el post-procesado no recupera son tan relevantes como las tablas de éxito. Ocultarlos habría sido más sencillo; mostrarlos hace el trabajo defendible y replicable.

13.5. Limitaciones y alcance del trabajo

Las limitaciones técnicas están en §12.4 (reproducibilidad). Aquí van las metodológicas, que son las que un tribunal preguntaría primero:

- **Tamaño del corpus de evaluación.** 4 BBDD × 3 runs × 4 experimentos × 3 modelos da una *n* teórica de 144 ontologías. Tras descartar fallos no recuperables y experimentos incompletos (E4 con open-source, calibración E3 con Llama), la *n* efectiva del análisis principal queda en

torno a 96. Es suficiente para tendencias robustas; insuficiente para tests estadísticos formales por celda.

- **Falta de ground truth.** Comparé las ontologías generadas entre sí con OQuaRE, pero no contra una ontología «correcta» del dominio. Sin un experto humano evaluando una muestra (~20 ontologías), no puedo decir si las que OQuaRE puntúa alto son biomédicamente correctas o simplemente bien estructuradas. La validación experta queda como trabajo futuro inmediato.
- **Sesgo de modelo cerrado.** La pipeline E4 (OntoGenix) está diseñada para gpt-4o y no la repliqué con éxito en open-source. Por tanto, parte de la conclusión sobre «el método compensa al modelo» (§10.6) descansa sobre 12 corridas con un único binomio. Replicar E4 con Llama 3.1 70B (no 8B) sería el siguiente paso lógico.
- **Drift de dependencias durante el TFM.** La integración de annotationRAG durante el desarrollo coincidió con un cambio de versión de numpy y torch que rompió varias librerías instaladas en el entorno principal. El experimento se completó, pero la documentación del estado de dependencias en cada ejecución (vía `check_reproducibility.py`) es retrospectiva, no longitudinal.

Potencia estadística declarada. El diseño con $n=3$ corridas por celda tiene una potencia limitada: solo diferencias OQuaRE iguales o superiores a 0.97 puntos se detectan al 80 % de potencia con $\alpha=0.05$ (ver Tabla 28 y Anexo B). Diferencias por debajo de este umbral se reportan como observacionales y no se interpretan como evidencia inferencial. Esto afecta especialmente a las comparaciones inter-modelo en E2 (donde el rango es ± 0.40 puntos) y a la diferencia entre la calibración C1 y C3, que queda dentro del ruido del propio muestreo.

13.6. Líneas de trabajo futuro

- **Validación experta sobre subset.** Tomar 20 ontologías estratificadas (5 por experimento) y pedir a un ontólogo biomédico que las evalúe contra criterios establecidos (BioPortal Quality Profile, NCBO Recommender). Comparar el ranking experto con el ranking OQuaRE. Si correlacionan, OQuaRE es operativo en el dominio; si no, hay que construir métrica específica.
- **Replicar E4 con Llama 3.1 70B.** La pipeline OntoGenix con un modelo open-source de tamaño comparable a gpt-4o permitiría aislar definitivamente el efecto «método» del efecto «modelo». Requiere una máquina con 64 GB+ de RAM o servicio remoto tipo Together o Groq.
- **Generalizar fuera del dominio cisreg.** El experimento más valioso para el campo sería repetir todo el banco con una fuente de datos no biomédica (clinical trials, drug-target, wine-vineyards) para ver si los hallazgos cualitativos se mantienen. Si el RAG semántico también añade +1 punto OQuaRE en esos dominios, la guía operativa de §10.7.7 se convierte en una recomendación general; si no, queda específica del dominio.

Ejecución del barrido del tamaño muestral (sugerencia 4 del director del TFM). El diseño completo se documenta en §10.9 con su tabla de coste asociada; los inputs (4 tamaños $\times$ 2 BBDD) están preparados en `data/samples_sizes/`, los scripts de ejecución y análisis están versionados (`scripts/run_size_sweep.sh` y `scripts/analyze_size_sweep.py`), y la estimación de coste indica \$0.86 USD para gpt-4o en E3 con RAG semántico. Es la línea de trabajo futuro de menor barrera de entrada y mayor probabilidad de aportar refinamiento empírico a las hipótesis H2 y H3.

14. Conclusiones

El presente trabajo ha desarrollado un banco experimental reproducible para evaluar cuatro estrategias de generación automática de ontologías biomédicas mediante modelos de lenguaje grandes, aplicado al dominio cis-regulatorio humano. Sobre 4 bases de datos, 3 modelos y 3 semillas, se generaron aproximadamente 240 ontologías evaluadas con dos métricas complementarias: OQuaRE estructural y fidelidad léxico-semántica al gold cisreg. La discusión detallada de los resultados se recoge en §13; este apartado resume los logros principales, la contribución científica del trabajo y sus limitaciones de alcance.

14.1. Logros principales

- Se ha verificado la hipótesis H1: las pipelines multi-agente (E4 OntoGenix) producen ontologías estructuralmente más ricas que las baselines de prompting directo, con una densidad de triples entre $3.91\times$ y $6.31\times$ superior (IC bootstrap, $p < 0.01$).
- Se ha verificado parcialmente la hipótesis H2: el RAG semántico real incrementa significativamente OQuaRE en gpt-4o (+1.29 puntos, +44 %), pero el efecto no es significativo en Llama 3.1 8B sin calibración del retrieval ($p = 0.07$).
- Se ha confirmado la hipótesis H3: la calibración del retrieval (top_k=2) cierra el 84 % del gap modelo-cerrado en Llama 8B, demostrando una interacción cuantitativamente medible entre tamaño del modelo y configuración del RAG.
- Se ha confirmado la hipótesis H4: el post-procesado mecánico determinista rescata el 100 % de las salidas sintácticamente recuperables en E3 (12/12, McNemar $p < 0.001$), separando el problema sintáctico del semántico.
- Se ha replicado funcionalmente la pipeline OntoGenix sobre las cuatro bases de datos con tres adaptaciones documentadas, incluyendo el soporte de modelos open-source vía Ollama.
- Se ha integrado un microservicio de RAG semántico real (annotationRAG) con índice vectorial Neo4j que sustituye al RAG por keyword-match del experimento original, mejorando la riqueza estructural pero reduciendo la fidelidad léxica.

14.2. Contribución principal

La contribución científica del trabajo se sintetiza en tres aportaciones replicables. Primero, una guía operativa cuantitativa para la calibración del RAG en función del tamaño del modelo, demostrando que la configuración óptima del retrieval depende de la capacidad de absorción de contexto del LLM y que la diferencia entre modelos cerrados y compactos puede mitigarse mediante una decisión de configuración, no de modelo. Segundo, un módulo de post-procesado mecánico determinista que separa el problema sintáctico del semántico y permite recuperar la totalidad de los outputs sintácticamente salvables sin invocaciones adicionales al modelo, aplicable a cualquier pipeline LLM $\rightarrow$ RDF. Tercero, una métrica complementaria de fidelidad léxico-semántica al dominio, calculable sin validación experta a partir de las ontologías de referencia del propio dominio, que matiza los resultados de OQuaRE y discrimina entre estrategias de reuso de vocabulario y estrategias de generación libre.

Adicionalmente, el trabajo aporta un marco de reproducibilidad alineado con la crisis de la replicación en investigación con LLMs, con manifest SHA-256 de inputs y dependencias, y una clasificación explícita de cada componente en tres niveles de garantía (R1 estricto, R2 estadístico, R3 metodológico).

14.3. Limitaciones globales

La principal limitación del trabajo es la ausencia de validación experta sobre la corrección biomédica del modelado. OQuaRE y la fidelidad léxica miden propiedades formales y de reuso, pero ninguna garantiza que las relaciones inferidas sean biológicamente correctas. El tamaño muestral (n=3 corridas por celda) limita la potencia estadística a detectar diferencias OQuaRE iguales o superiores a 0.97 puntos (§9.4); diferencias menores se reportan como observacionales. La pipeline OntoGenix con modelos open-source de tamaño comparable a gpt-4o no se ha completado por restricciones de hardware. Estas limitaciones se discuten con detalle en §13.5 junto con las líneas de trabajo futuro propuestas en §13.6.

15. Referencias (APA 7^a)

- Val Calvo, M., Egaña Aranguren, M., & Fernández Breis, J. T. (2024). OntoGenix: Leveraging large language models for enhanced ontology engineering from datasets. *Information Processing & Management*, 62(3), 104042. <https://doi.org/10.1016/j.ipm.2024.104042>
- Meyer, L.-P., Stadler, C., Frey, J., Radtke, N., Junghanns, K., Meissner, R., Dziwis, G., Bulert, K., & Martin, M. (2023). LLM-assisted knowledge graph engineering: Experiments with ChatGPT. *arXiv:2307.06917*. <https://arxiv.org/abs/2307.06917>
- Neuhaus, F. (2023). Ontologies in the era of large language models – A perspective. *Applied Ontology*, 18(4), 399–407. <https://doi.org/10.3233/AO-230072>
- Studer, R., Benjamins, V. R., & Fensel, D. (1998). Knowledge engineering: Principles and methods. *Data & Knowledge Engineering*, 25(1–2), 161–197. [https://doi.org/10.1016/S0169-023X\(97\)00056-6](https://doi.org/10.1016/S0169-023X(97)00056-6)
- W3C OWL Working Group. (2012). OWL 2 Web Ontology Language Document Overview (Second Edition). W3C Recommendation. <https://www.w3.org/TR/owl2-overview/>
- Andersson, R., Gebhard, C., Miguel-Escalada, I., Hoof, I., Bornholdt, J., Boyd, M., Chen, Y., Zhao, X., Schmidl, C., Suzuki, T., & FANTOM Consortium. (2014). An atlas of active enhancers across human cell types and tissues. *Nature*, 507(7493), 455–461. <https://doi.org/10.1038/nature12787>
- Khan, A., & Zhang, X. (2016). dbSUPER: A database of super-enhancers in mouse and human genome. *Nucleic Acids Research*, 44(D1), D164–D171. <https://doi.org/10.1093/nar/gkv1002>
- Wang, J., Dai, X., Berry, L. D., Cogan, J. D., Liu, Q., & Shyr, Y. (2019). HACER: An atlas of human active enhancers to interpret regulatory variants. *Nucleic Acids Research*, 47(D1), D106–D112. <https://doi.org/10.1093/nar/gky864>

- Zhang, G., Shi, J., Zhu, S., Lan, Y., Xu, L., Yuan, H., Liao, G., Liu, X., Zhang, Y., Xiao, Y., & Li, X. (2018). DiseaseEnhancer: A resource of human disease-associated enhancer catalog. *Nucleic Acids Research*, 46(D1), D78–D84. <https://doi.org/10.1093/nar/gkx920>
- OpenAI. (2024). GPT-4o system card. <https://openai.com/index/gpt-4o-system-card/>
- Duque-Ramos, A., Fernández-Breis, J. T., Stevens, R., & Aussenac-Gilles, N. (2011). OQuaRE: A SQuaRE-based approach for evaluating the quality of ontologies. *Journal of Research and Practice in Information Technology*, 43(2), 159–176.
- Duque-Ramos, A., Boeker, M., Jansen, L., Schulz, S., Iniesta, M., & Fernández-Breis, J. T. (2014). Evaluating the good ontology design guideline (GoodOD) with the ontology quality requirements and evaluation method and metrics (OQuaRE). *PLoS ONE*, 9(8), e104463. <https://doi.org/10.1371/journal.pone.0104463>
- Glimm, B., Horrocks, I., Motik, B., Stoilos, G., & Wang, Z. (2014). HermiT: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3), 245–269. <https://doi.org/10.1007/s10817-014-9305-1>
- Lamy, J. B. (2017). Owlready: Ontology-oriented programming in Python with automatic classification and high level constructs for biomedical ontologies. *Artificial Intelligence in Medicine*, 80, 11–28. <https://doi.org/10.1016/j.artmed.2017.07.002>
- Meta AI. (2024). The Llama 3 herd of models. <https://ai.meta.com/research/publications/the-llama-3-herd-of-models/>
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., & Lin, J. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186. <https://arxiv.org/abs/2409.12186>
- Ollama. (2024). Ollama: Get up and running with large language models locally [Software]. <https://ollama.com>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3982–3992. <https://doi.org/10.18653/v1/D19-1410>
- Tecnomod (Universidad de Murcia). (2024). annotationRAG: A Retrieval-Augmented Generation approach for the automated annotation of biological sample metadata [Software]. GitHub. <https://github.com/tecnomod-um/annotationRAG>
- Neo4j Inc. (2024). Vector indexes — Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/>

- Open Science Collaboration. (2015). Estimating the reproducibility of psychological science. *Science*, 349(6251), aac4716. <https://doi.org/10.1126/science.aac4716>
- Hutson, M. (2018). Artificial intelligence faces reproducibility crisis. *Science*, 359(6377), 725–726. <https://doi.org/10.1126/science.359.6377.725>
- Gundersen, O. E., & Kjensmo, S. (2018). State of the art: Reproducibility in artificial intelligence. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://doi.org/10.1609/aaai.v32i1.11503>
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018). Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). <https://doi.org/10.1609/aaai.v32i1.11694>
- Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164), 1–20. <https://jmlr.org/papers/v22/20-303.html>
- Goodman, S. N., Fanelli, D., & Ioannidis, J. P. (2016). What does research reproducibility mean? *Science Translational Medicine*, 8(341), 341ps12. <https://doi.org/10.1126/scitranslmed.aaf5027>
- Reynolds, L., & McDonell, K. (2021). Prompt programming for large language models: Beyond the few-shot paradigm. *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–7. <https://doi.org/10.1145/3411763.3451760>
- (Las siguientes referencias se han añadido como resultado de la ampliación del Estado del arte y la metodología en respuesta a la revisión del director del TFM.)
- Andersson, R., Gebhard, C., Miguel-Escalada, I., Hoof, I., Bornholdt, J., Boyd, M., et al. (2014). An atlas of active enhancers across human cell types and tissues. *Nature*, 507(7493), 455–461. <https://doi.org/10.1038/nature12787>
- Bandeira, J., Bittencourt, I. I., Espinheira, P., & Isotani, S. (2016). FOCA: A methodology for ontology evaluation. *arXiv preprint arXiv:1612.03353*.
- Begley, C. G., & Ellis, L. M. (2012). Drug development: Raise standards for preclinical cancer research. *Nature*, 483(7391), 531–533. <https://doi.org/10.1038/483531a>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Caufield, J. H., Hegde, H., Emonet, V., Harris, N. L., Joachimiak, M. P., Matentzoglou, N., et al. (2023). SPIRES: Structured Prompt Interrogation and Recursive Extraction of Semantics — A method for populating knowledge bases using zero-shot learning. *arXiv preprint arXiv:2304.02711*.
- Caufield, J. H., Hegde, H., Emonet, V., Harris, N. L., Joachimiak, M. P., Matentzoglou, N., et al. (2024). Structured prompt interrogation and recursive extraction of semantics (SPIRES): A method for

populating knowledge bases using zero-shot learning. *Bioinformatics*, 40(3), btae104.

<https://doi.org/10.1093/bioinformatics/btae104>

Cuenca-Grau, B., Horrocks, I., Motik, B., Parsia, B., Patel-Schneider, P., & Sattler, U. (2008). OWL 2: The next step for OWL. *Journal of Web Semantics*, 6(4), 309–322.

Glimm, B., Horrocks, I., Motik, B., Stoilos, G., & Wang, Z. (2014). HermiT: An OWL 2 reasoner. *Journal of Automated Reasoning*, 53(3), 245–269. <https://doi.org/10.1007/s10817-014-9305-1>

Grüninger, M., & Fox, M. S. (1995). Methodology for the design and evaluation of ontologies. *Proceedings of the IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing*.

Gu, Y., Tinn, R., Cheng, H., Lucas, M., Usuyama, N., Liu, X., et al. (2021). Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare*, 3(1), 1–23.

Khan, A., & Zhang, X. (2016). dbSUPER: A database of super-enhancers in mouse and human genome. *Nucleic Acids Research*, 44(D1), D164–D171. <https://doi.org/10.1093/nar/gkv1002>

Kommineni, V. K., König-Ries, B., & Samuel, S. (2024). From human experts to machines: An LLM-supported approach to ontology and knowledge graph construction. *arXiv preprint arXiv:2403.08345*.

Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., & Kang, J. (2020). BioBERT: A pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4), 1234–1240.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Liu, F., Shareghi, E., Meng, Z., Basaldella, M., & Collier, N. (2021). Self-alignment pretraining for biomedical entity representations. *Proceedings of NAACL-HLT 2021*, 4228–4238.

Martínez-Romero, M., Jonquet, C., O'Connor, M. J., Graybeal, J., Pazos, A., & Musen, M. A. (2017). NCBO Ontology Recommender 2.0: An enhanced approach for biomedical ontology recommendation. *Journal of Biomedical Semantics*, 8(1), 21.

Mateiu, P. M., & Groza, A. (2023). Ontology engineering with large language models. *Procedia Computer Science*, 225, 1095–1104.

Meyer, L. P., Stadler, C., Frey, J., Radtke, N., Junghanns, K., Meissner, R., et al. (2023). LLM-assisted knowledge graph engineering: Experiments with ChatGPT. *arXiv preprint arXiv:2307.06917*.

Motik, B., Patel-Schneider, P. F., & Parsia, B. (Eds.). (2012). *OWL 2 Web Ontology Language: Structural specification and functional-style syntax* (2nd ed.). W3C Recommendation.

Norouzi, S. S., Mahdavinjad, M. S., & Hitzler, P. (2024). Conversational ontology alignment with ChatGPT. *arXiv preprint arXiv:2308.09217*.

- Ochs, C., Geller, J., Perl, Y., Chen, Y., Xu, J., Min, H., & Case, J. T. (2017). Scalable quality assurance for large SNOMED CT hierarchies using subject-based subtaxonomies. *Journal of the American Medical Informatics Association*, 24(3), 524–533.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *Proceedings of EMNLP 2019*, 3982–3992.
- Tartir, S., Arpinar, I. B., Moore, M., Sheth, A. P., & Aleman-Meza, B. (2005). OntoQA: Metric-based ontology quality analysis. *IEEE ICDM Workshop on Knowledge Acquisition from Distributed, Autonomous, Semantically Heterogeneous Data and Knowledge Sources*.
- Vrandečić, D. (2010). Ontology evaluation. In S. Staab & R. Studer (Eds.), *Handbook on ontologies* (pp. 293–313). Springer.
- W3C. (2014). RDF 1.1 Turtle: Terse RDF Triple Language. W3C Recommendation. <https://www.w3.org/TR/turtle/>
- Wang, J., Dai, X., Shen, X., Ge, X., Wei, B., Tian, R., et al. (2019). HACER: An atlas of human active enhancers to interpret regulatory variants. *Nucleic Acids Research*, 47(D1), D106–D112.
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained Transformers. *Advances in Neural Information Processing Systems*, 33, 5776–5788.
- Zhang, G., Shi, J., Zhu, S., Lan, Y., Xu, L., Li, H., et al. (2018). DiseaseEnhancer: A resource of human disease-associated enhancer catalog. *Nucleic Acids Research*, 46(D1), D78–D84.

Anexo A. Abreviaturas y siglas

Listado alfabético de las abreviaturas, siglas y acrónimos empleados en el documento. Cuando un término aparece por primera vez en el texto, se introduce con su forma desarrollada y, entre paréntesis, su sigla.

Sigla	Definición
ABox	Assertional Box — conjunto de instancias en una ontología.
ANOnto	Annotation Number Ontology (métrica OQuaRE de riqueza documental).
APA	American Psychological Association (estilo de referencias).
API	Application Programming Interface.
AROnto	Attribute Richness Ontology (métrica OQuaRE).
BBDD	Bases de datos.
BioPortal	Repositorio público de ontologías biomédicas mantenido por NCBO.
CAGE	Cap Analysis of Gene Expression.

CBOnto	Coupling Between Ontologies (métrica OQuaRE de modularidad).
CRM	Cis-Regulatory Module (elemento regulador en cis).
CSV	Comma-Separated Values.
dbSUPER	Database of Super-Enhancers.
DITM	Depth of Inheritance Tree Modularity (métrica OQuaRE).
FANTOM5	Functional Annotation of the Mammalian Genome 5.
FastAPI	Framework Python para construcción de APIs HTTP.
GPU	Graphics Processing Unit.
HACER	Human ACtive Enhancer to gene Regulation database.
HermiT	Razonador OWL DL utilizado vía owlready2.
HTTP	HyperText Transfer Protocol.
INROnto	Inverse Relations Ontology (métrica OQuaRE).
IRI	Internationalized Resource Identifier.
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission.
JSON-LD	JSON for Linking Data — serialización de RDF (W3C).
LCOMOnto	Lack of Cohesion in Methods Ontology (métrica OQuaRE).
LLM	Large Language Model — modelo de lenguaje grande.
MCP	Model Context Protocol.
MIA	Máster Universitario en Inteligencia Artificial (UMU).
NACOnto	Number of Ancestor Classes Ontology (métrica OQuaRE).
NCBO	National Center for Biomedical Ontology.
Neo4j	Base de datos orientada a grafos con índice vectorial nativo.
NOMOnto	Number Of Methods Ontology (métrica OQuaRE).
Ollama	Servidor local para modelos LLM open-source.
OntoGenix	Pipeline multi-agente para generación de ontologías (Val Calvo et al., 2024).
OQuaRE	Ontology Quality Requirements and Evaluation — marco normativo basado en

	ISO/IEC 25000.
OWL	Web Ontology Language (recomendación W3C).
OWL DL	Subconjunto Description Logic de OWL.
owlready2	Biblioteca Python para ontologías OWL con integración de razonadores.
RAG	Retrieval-Augmented Generation — generación aumentada por recuperación.
RAM	Random Access Memory.
RDF	Resource Description Framework (W3C).
RDF/XML	Serialización XML de RDF.
rdflib	Biblioteca Python para manipular grafos RDF.
RDFS	RDF Schema (W3C).
RROnto	Relationship Richness Ontology (métrica OQuaRE).
SERP	Search Engine Results Page.
SHA-256	Secure Hash Algorithm de 256 bits.
SHACL	Shapes Constraint Language (W3C).
SPARQL	SPARQL Protocol And RDF Query Language.
SQuaRE	Systems and software Quality Requirements and Evaluation (ISO/IEC 25000).
TBox	Terminological Box — clases y propiedades en una ontología.
TF	Transcription Factor (factor de transcripción).
TFM	Trabajo Fin de Máster.
TMOnto	Tangledness Modularity Ontology (métrica OQuaRE).
TSS	Transcription Start Site.
TSV	Tab-Separated Values.
TTL	Turtle — Terse RDF Triple Language.
UMU	Universidad de Murcia.
URI	Uniform Resource Identifier.
WMCOnto	Weighted Methods Count Ontology (métrica OQuaRE).

XSD	XML Schema Definition.
-----	------------------------

Las métricas OQuaRE incluidas (WMCOnto, NOMOnto, DITM, NACOnto, CBOnto, TMOnto, LCOMOnto, ANOnto, INROnto, RROnto y AROnto) se documentan con su rúbrica completa en la Tabla 12 del cuerpo de la memoria.

Anexo B. Reporte completo de tests estadísticos

Este anexo reproduce los resultados del módulo `scripts/statistical_tests.py` aplicado a los CSV crudos del experimento (`results/comparison_E1-E4.csv` para métricas absolutas y `results/evaluation/oquare_metrics.csv` para la calibración del RAG en Llama 3.1 8B). Todos los bootstrap usan 10 000 réplicas con semilla fija (42). El reporte completo en formato Markdown está disponible en `results/evaluation/statistical_tests.md`.

B.1. Densidad estructural — E4 vs E1/E2/E3 (n_triples)

Bootstrap pareado de la diferencia de medias y Wilcoxon de rangos con signo sobre 12 pares (4 BBDD × 3 corridas).

Comparación	n pares	Δ media [IC 95 %]	Cohen's d	Wilcoxon p	Ratio E4/Base
E4 vs E1	12	+174.7 [+164.6, +186.7]	8.48 (grande)	0.002	4.51× (3.08–6.03)
E4 vs E2	12	+159.9 [+129.4, +183.8]	3.23 (grande)	0.002	3.91× (1.11–7.03)
E4 vs E3	12	+184.3 [+169.5, +201.8]	6.15 (grande)	0.002	6.31× (2.76–12.77)

B.2. Riqueza documental — E4 vs E1/E2/E3 (n_labels)

Comparación	n pares	Δ media [IC 95 %]	Cohen's d
E4 vs E1	12	+43.1 [+41.8, +44.1]	20.43
E4 vs E2	12	+42.4 [+41.3, +43.3]	23.15
E4 vs E3	12	+43.1 [+41.8, +44.1]	20.43

B.3. Validez sintáctica antes/después del post-procesado (McNemar)

Experimento	n	parse_ok raw	parse_ok post	Rescatadas	Regresadas	McNemar p
E1	12	83 %	100 %	2	0	0.500
E2	12	67 %	100 %	4	0	0.125

E3	12	0 %	100 %	12	0	<0.001
----	----	-----	-------	----	---	--------

B.4. Calibración del RAG en Llama 3.1 8B

Pareo estricto por (BBDD, semilla) condicional a load_ok=1 en ambas configuraciones.

Variante	n pares	Δ OQuaRE [IC 95 %]	Cohen's d	p ajustado (Holm)
legacy vs ragapi	6	-0.23 [-0.46, +0.01]	-0.68 (mediano)	0.070
C1 (top_k=2) vs ragapi	2	+0.80 [+0.80, +0.80]	—	<0.001
C2 (thr=0.6) vs ragapi	2	+0.43 [+0.35, +0.50]	+4.01 (grande)	<0.001
C3 (combinada) vs ragapi	3	+0.49 [+0.10, +0.80]	+1.37 (grande)	<0.001

B.5. Gap respecto a gpt-4o (referencia OQuaRE = 4.20)

Variante	n_runs OK	Media $\pm$ SD	Gap a gpt-4o	% gap cerrado
llama3.1_8b_legacy	12	3.45 $\pm$ 0.05	+0.75	0.0 %
llama3.1_8b_ragapi	6	3.69 $\pm$ 0.31	+0.51	31.3 %
llama3.1_8b_ragapi_C1	6	4.08 $\pm$ 0.13	+0.12	83.9 %
llama3.1_8b_ragapi_C2	5	3.74 $\pm$ 0.80	+0.46	38.4 %
llama3.1_8b_ragapi_C3	7	3.49 $\pm$ 1.13	+0.71	5.1 %

B.6. Análisis de potencia para OQuaRE Global

SD muestral observada en oquare_global: 0.602.

n por celda	Δ OQuaRE detectable	Cohen's d equivalente	Interpretación
n = 3	0.974	1.62	grande
n = 6	0.689	1.14	grande
n = 12	0.487	0.81	grande
n = 24	0.344	0.57	mediano

B.7. Corrección Bonferroni-Holm de p-valores principales

Test	p crudo	p ajustado (Holm)	Sig. ($\alpha = 0.05$)
density E4 vs E1	0.002	0.009	✓
density E4 vs E2	0.002	0.009	✓

density E4 vs E3	0.002	0.009	✓
calibration legacy vs ragapi	0.070	0.070	—
calibration C1 vs ragapi	<0.001	<0.001	✓
calibration C2 vs ragapi	<0.001	<0.001	✓
calibration C3 vs ragapi	<0.001	<0.001	✓

Las cuatro primeras tablas (B.1–B.4) reproducen los tests primarios y son consistentes con la Tabla 27 del cuerpo. La Tabla B.5 contextualiza la calibración respecto al modelo cerrado de referencia. La Tabla B.6 fija el techo inferencial del diseño ($n=3$ detecta diferencias ≥ 0.97 OQuaRE). La Tabla B.7 muestra que las cuatro afirmaciones primarias sobre densidad de E4 y las tres configuraciones de calibración resisten la corrección por comparaciones múltiples; la diferencia legacy $\rightarrow$ ragapi en Llama queda por encima del umbral $\alpha=0.05$ y se reporta como observacional.

Anexo C. Plantillas de prompts utilizadas

Este anexo reproduce las plantillas de prompts utilizadas en las cuatro estrategias de generación. Las variables entre llaves se sustituyen en tiempo de ejecución por los valores correspondientes a cada (BBDD, run). La descripción funcional de cada estrategia está en §3.4.

La totalidad de las indicaciones utilizadas se listan a continuación. Todas comparten un mismo system prompt que fija el rol del modelo como ontólogo biomédico.

C.1. System prompt común (E1–E3)

You are an expert ontologist specializing in biological knowledge graphs and semantic web technologies.
Your expertise includes OWL 2, RDF, SPARQL, and biological domain ontologies (SO, GO, BioLink Model, SKOS).
Your task is to analyze biological database samples and generate formal OWL ontology schemas in Turtle serialization.

Domain context: The databases contain information about cis-regulatory modules (CRM), specifically enhancer sequences that modulate the transcription rate of their target genes in the human genome. These sequences are characterized by their genomic coordinates, experimental evidence, cell line activity, target genes, associated diseases, and interacting transcription factors.

C.2. Indicación E1 — Zero-shot

Analyze the following biological database sample and generate a formal OWL ontology schema in Turtle format.

DATABASE: {db_name}
DESCRIPTION: {db_description}

DATABASE SAMPLE (TSV format, "-" means missing value):
{db_sample}

Think step by step:

1. Identify the main concepts/classes present in the data
2. Identify the relationships (object properties) and attributes (datatype properties)
3. Map concepts to standard biological ontologies where possible (SO, GO, BioLink)
4. Generate the formal Turtle schema

Requirements for the output schema:

- Use these standard prefixes: rdf:, rdfs:, owl:, skos:, xsd:, obo: (<http://purl.obolibrary.org/obo/>)
- Add skos:prefLabel and skos:definition to all classes and properties
- Include at least: class for CRM/enhancer, genomic coordinate properties, evidence properties
- Use owl:Class for all main concepts
- Use owl:ObjectProperty for relationships and owl:DatatypeProperty for literals
- Include skos:exactMatch or skos:closeMatch mappings to standard biological ontologies where appropriate

Output ONLY valid Turtle code. No explanations, no markdown code blocks.

C.3. Indicación E2 — Vocabulario controlado

Analyze the following biological database sample and generate a formal OWL ontology schema in Turtle format.

Use the provided controlled vocabulary to name and align your ontology terms.

DATABASE: {db_name}
DESCRIPTION: {db_description}

DATABASE SAMPLE (TSV format, "-" means missing value):
{db_sample}

CONTROLLED VOCABULARY (cisreg terms):
{vocab_terms}

Think step by step:

1. Identify the main concepts/classes in the data
2. Map them to the controlled vocabulary terms above
3. Identify relationships and attributes, reusing vocabulary terms
4. Generate the formal Turtle schema

Requirements:

- Prefer vocabulary terms for class and property names where applicable
- Use standard prefixes: rdf:, rdfs:, owl:, skos:, xsd:, obo:
- Include skos:prefLabel, skos:definition, and skos:exactMatch/closeMatch mappings

- Include both owl:Class and owl:ObjectProperty/owl:DatatypeProperty declarations

Output ONLY valid Turtle code. No explanations, no markdown code blocks.

El bloque {vocab_terms} se construye automáticamente a partir de los esquemas cisreg (crm, crm2gene, crm2phen y crm2tfac), extrayendo hasta 80 líneas con skos:prefLabel o skos:definition. De ese modo se obtiene un vocabulario compacto y relevante para el dominio, sin incluir la estructura axiomática completa de la ontología.

C.4. Indicación E3 — RAG (fragmentos ontológicos)

Analyze the following biological database sample and generate a formal OWL ontology schema in Turtle format. Align your schema as closely as possible with the cisreg reference ontology fragments provided below.

DATABASE: {db_name}
DESCRIPTION: {db_description}

DATABASE SAMPLE (TSV format, "-" means missing value):
{db_sample}

CISREG REFERENCE ONTOLOGY (most relevant fragments in Turtle):
{ontology_fragments}

Think step by step:

1. Identify the main concepts/classes in the data
2. Find the corresponding classes and properties in the cisreg reference ontology
3. Reuse cisreg URIs where semantically appropriate
4. Add new terms only where cisreg does not cover the concept

Requirements:

- Reuse cisreg classes and properties where semantically appropriate
- Add explicit skos:exactMatch or skos:closeMatch to cisreg URIs for aligned terms
- Use all standard prefixes from the cisreg ontology
- New classes/properties only where cisreg doesn't cover the concept

Output ONLY valid Turtle code. No explanations, no markdown code blocks.

El bloque {ontology_fragments} se recupera mediante una heurística que selecciona los esquemas cisreg relevantes según las palabras clave presentes en la muestra (crm siempre, crm2gene si aparece 'gene', crm2phen si aparece 'disease'/'phenotype', crm2tfac si aparece 'transcription factor').

C.5. Indicación E4 — OntoGenix

OntoGenix utiliza tres prompts externos ubicados en OntoGenix/prompts/ontogenix/. Cada prompt incluye placeholders que se rellenan en tiempo de ejecución con los artefactos generados en pasos previos. La tabla siguiente resume el rol de cada agente:

Tabla 8. Roles de los tres agentes de la pipeline OntoGenix

El paso adicional LlmPlanner.update(), previsto para enriquecer el razonamiento con resultados de schema.org vía SERP API, se omite en este TFM por política de claves externas. La ontología se genera directamente a partir de LlmPlanner.interaction().

Agente	Prompt base	Rol
LlmPlanner	planner_instructions.txt	Analiza las estadísticas del CSV y propone qué clases y propiedades necesita la ontología; genera un razonamiento en markdown
LlmOntology	ontology_instructions.txt	A partir del razonamiento y las estadísticas, genera una ontología OWL 2 en Turtle
LlmOntoMapper	ontomapper_instructions.txt	A partir de la ontología y el CSV, produce un mapping (RML o TTL) que permite materializar RDF desde los datos

Anexo D. Corpus de preguntas de competencia

Listado completo de las 15 preguntas de competencia utilizadas en la evaluación funcional (§9.6). Cada pregunta incluye su formulación en lenguaje natural y la consulta SPARQL canónica. El criterio de satisfacción TBox y los patrones de requisitos por pregunta están documentados en `scripts/competency_questions.py`.

CQ01. ¿Qué módulo cis-regulador (CRM) está localizado en una región genómica dada (cromosoma, start, end)?

```
SELECT ?crm WHERE {
  ?crm a hcrm:crm_ID ;
  obo:BFO_0000050 ?chr ;
  obo:GENO_0000895 ?start ;
  obo:GENO_0000894 ?end .
  FILTER(?chr = nuccore:chr1 && ?start >= 1000 && ?end <= 2000) }
```

CQ02. ¿Qué genes diana regula un CRM concreto?

```
SELECT ?gene WHERE {
  ?crm a hcrm:crm_ID ;
  sio:SIO_000628 ?gene .
  FILTER(?crm = hcrm:CRMHS00000005752) }
```

CQ03. ¿Qué factores de transcripción se unen a un CRM?

```
SELECT ?tf WHERE {
  ?crm a hcrm:crm_ID ;
  obo:RO_0002436 ?tf .
  ?tf a sio:SIO_010035 . }
```

CQ04. ¿Qué enhancers están asociados a una enfermedad concreta?

```
SELECT ?crm WHERE {
  ?crm a hcrm:crm_ID ;
  obo:RO_0004026 ?disease .
  FILTER(?disease = obo:DOID_0060785) }
```

CQ05. ¿Cuál es la evidencia experimental que respalda un CRM?

```
SELECT ?evidence ?article WHERE {
  ?crm a hcrm:crm_ID ;
  rdfs:isDefinedBy ?evidence ;
  sio:SIO_000772 ?article . }
```

CQ06. ¿En qué tejido o tipo celular es activo un CRM?

```
SELECT ?tissue WHERE {
  ?crm a hcrm:crm_ID ;
  obo:TXPO_0003500 ?tissue .
  ?tissue a obo:UBERON_ID . }
```

CQ07. ¿Qué super-enhancers están registrados en dbSUPER y cuáles son sus genes diana?

```
SELECT ?se ?gene WHERE {
  ?se a hcrm:crm_ID ;
  rdfs:isDefinedBy <dbSUPER> ;
  sio:SIO_000628 ?gene . }
```

CQ08. ¿Qué mutaciones se han documentado en un CRM?

```
SELECT ?mutation WHERE { ?crm a hcrm:crm_ID ; obo:RO_0001025 ?mutation . }
```

CQ09. ¿Qué versión de ensamblaje genómico (hg19 / hg38) utiliza la anotación de un CRM?

```
SELECT ?assembly WHERE { ?crm a hcrm:crm_ID ; dc:hasVersion ?assembly . }
```

CQ10. ¿Cuál es el método experimental utilizado para identificar un enhancer (CAGE, H3K27ac, etc.)?

```
SELECT ?method WHERE { ?crm a hcrm:crm_ID ; obo:OBI_0000293 ?method . }
```

CQ11. ¿En qué taxón (especie) se encuentra anotado un CRM?

```
SELECT ?taxon WHERE { ?crm a hcrm:crm_ID ; obo:RO_0002162 ?taxon . }
```

CQ12. ¿Qué CRM tienen un score de confianza mínimo determinado?

```
SELECT ?crm ?score WHERE { ?crm a hcrm:crm_ID ; sio:SIO_000300 ?score . FILTER(?score >= 0.9) }
```

CQ13. ¿Cuál es el cross-reference (identificador externo) de un enhancer en su base de datos original?

```
SELECT ?xref WHERE { ?crm a hcrm:crm_ID ; sio:SIO_000253 ?xref . }
```

CQ14. ¿Existe una relación CRM → fenotipo distinguible de CRM → enfermedad?

```
SELECT ?phenotype WHERE { ?crm a hcrm:crm_ID ; obo:RO_0002200 ?phenotype . ?phenotype a obo:HP_0000118 . }
```

CQ15. ¿Hay anotación bibliográfica (PMID) que valide la asociación CRM → gen para una corrida concreta?

```
SELECT ?article WHERE { ?crm a hcrm:crm_ID ; sio:SIO_000628 ?gene ; sio:SIO_000772 ?article . ?article a obo:IAO_0000013 . }
```